

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1. СИСТЕМНЫЙ АНАЛИЗ И ПОСТАНОВКА ЗАДАЧИ	7
1.1 Основные понятия предметной области	7
1.2 Обследование процесса мерчендайзинга и выявление проблем	9
1.3 Введение в компьютерное зрение, нейронные сети и машинное обучение	12
1.4 Требования к системе «Digital Shelf» и постановка задачи	15
2. АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ	17
2.1 Обзор существующих решений и обоснование разработки	17
2.2 Архитектура YOLOv8 и её применимость к задаче	17
2.3 Архитектура Telegram-бота на базе aiogram	19
2.4 Оценка соответствия выкладки планограмме	20
3. СБОР И ПОДГОТОВКА ДАТАСЕТА ПРОДУКЦИИ САНТА ИМПЭКС	22
3.1 Анализ и структура собираемых данных	22
3.2 Разметка данных и формирование датасета	24
3.3 Обработка данных и аугментация	28
4. РАЗРАБОТКА, ОБУЧЕНИЕ И ОЦЕНКА МОДЕЛИ НЕЙРОННОЙ СЕТИ	30
4.1 Выбор архитектуры нейронной сети для детекции объектов	30
4.2 Обзор архитектуры YOLOv8	30
4.3 Обучение модели YOLOv8	32
4.4 Гиперпараметры модели YOLOv8	33
4.5 Разработка мобильного приложения «Retail Vision»	34
4.6 Тестирование и оценка модели нейронной сети	37
5. РАЗВЁРТЫВАНИЕ СИСТЕМЫ «RETAIL VISION»	40
5.1 Выбор стратегии развёртывания системы	40
5.2 Конвертация и оптимизация модели YOLOv8n для развёртывания	42
5.3 Развёртывание Telegram-бота на базе aiogram	43
5.4 Интеграция модуля сравнения с планограммой	46
5.5 Контейнеризация и конфигурация серверного окружения	47
5.6 Тестирование развёрнутой системы	48
5.7 Безопасность развёртывания и управление секретами	49
5.8 Мониторинг, логирование и оповещение	50
5.9 Резервное копирование и восстановление планограмм	51
6. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ	53
6.1 Исходные данные для расчета экономического эффекта	53
6.2 Расчет объема функций программного обеспечения	53
6.3 Расчет полной себестоимости программного продукта	54
6.4 Расчет цены и прибыли по программному продукту	58
ЗАКЛЮЧЕНИЕ	61
СПИСОК ЛИТЕРАТУРЫ	62
ПРИЛОЖЕНИЕ А Описание применения	
ПРИЛОЖЕНИЕ Б Текст программы	

ВВЕДЕНИЕ

Современная розничная торговля предъявляет высокие требования к качеству выкладки товаров на полках магазинов. Соблюдение планограмм — утверждённых схем расположения продукции — напрямую влияет на объём продаж, восприятие бренда покупателем и выполнение контрактных обязательств между производителем и торговой сетью. Компания «Санта Импэкс», являясь дистрибьютором широкого ассортимента потребительских товаров, ежедневно направляет мерчендайзеров в сотни торговых точек для контроля выкладки. При этом процесс проверки по-прежнему остаётся преимущественно ручным: сотрудник визуально оценивает полку, делает фотографию и вручную заполняет отчёт, что влечёт за собой высокую трудоёмкость, субъективность оценки и значительные временные затраты.

Актуальность настоящей работы обусловлена необходимостью автоматизации контроля мерчендайзинга с применением технологий компьютерного зрения и глубокого обучения. Развитие нейросетевых архитектур обнаружения объектов, в частности семейства YOLO (You Only Look Once), позволяет в режиме реального времени анализировать фотографии торговых полок, идентифицировать товарные позиции и сопоставлять их расположение с эталонной планограммой. Интеграция подобной системы с мессенджером Telegram даёт возможность мерчендайзеру использовать привычный инструмент без установки дополнительного программного обеспечения на рабочее устройство.

Целью дипломного проекта является разработка программной системы автоматизированной проверки выкладки товаров, включающей Telegram-бот для полевых сотрудников и нейросетевой модуль на основе архитектуры YOLOv8, обеспечивающий распознавание продукции и оценку соответствия планограмме.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить предметную область мерчендайзинга и требования компании «Санта Импэкс» к контролю выкладки;
- провести обзор существующих аналогов систем компьютерного зрения для ритейла;
- спроектировать архитектуру системы: Telegram-бот, нейросетевой модуль обнаружения объектов и базу данных хранения результатов;
- обучить модель YOLOv8 на размеченном наборе данных фотографий торговых полок с продукцией компании;
- реализовать алгоритм сопоставления результатов детекции с планограммой и генерации аннотированного изображения;
- разработать механизм автоматического формирования отчётов о корректности выкладки и их сохранения в базе данных;
- провести тестирование системы и оценить метрики качества работы нейронной сети;
- выполнить технико-экономическое обоснование разработки.

Практическая значимость работы для компании «Санта Импэкс» определяется тремя ключевыми эффектами. Во-первых, автоматизация проверки позволяет сократить время, затрачиваемое мерчендайзером на оформление результатов визита: сотрудник отправляет фотографию в бот и получает заключение о соответствии выкладки. Во-вторых, автоматическое формирование отчётов и их централизованное хранение создаёт базу для анализа качества выкладки и оперативного реагирования на нарушения. В-третьих, снижение доли ручного труда уменьшает операционные затраты и позволяет эффективнее использовать рабочее время сотрудников.

1. СИСТЕМНЫЙ АНАЛИЗ И ПОСТАНОВКА ЗАДАЧИ

1.1 Основные понятия предметной области

Объектом исследования и автоматизации в рамках данной работы является процесс мерчендайзинга продукции компании «Санта Импэкс» в торговых точках. Для понимания предметной области необходимо разобрать ключевые понятия, связанные с выкладкой товаров, стандартами размещения продукции, а также определить, каким образом методы компьютерного зрения и глубокого обучения могут быть применены для автоматизации контроля соблюдения этих стандартов.

Процесс контроля выкладки базируется на строгом соблюдении планограмм — утвержденных схем расположения товаров, которые определяют количество «фейсингов» (единиц товара, обращенных к покупателю) и их последовательность на полке. В настоящее время аудит этих стандартов выполняется мерчендайзерами вручную, что влечет за собой значительные временные затраты и риск возникновения ошибок, обусловленных человеческим фактором. Ручной сбор данных часто приводит к субъективности оценок, а задержка в передаче отчетов в центральный офис компании не позволяет оперативно реагировать на нарушения стандартов или отсутствие продукции в торговом зале.

Автоматизация данного процесса с использованием глубоких нейронных сетей позволяет перевести визуальный аудит в плоскость объективного цифрового анализа. Применение современных архитектур детекторов объектов, в частности семейства моделей YOLO (You Only Look Once), дает возможность системе одновременно локализовать каждую единицу продукции на снимке и классифицировать её принадлежность к конкретному SKU. Благодаря обучению на размеченном наборе данных, нейросеть извлекает высокоуровневые признаки объектов, что гарантирует стабильную работу системы даже в сложных условиях: при недостаточном освещении в магазине, наличии бликов на упаковке или частичном перекрытии одного товара другим.

Итоговой целью внедрения разрабатываемой системы является создание единого аналитического контура, связывающего мобильное приложение мерчендайзера и облачную базу данных компании. Автоматическое распознавание товаров на полке не только сокращает время аудита одной торговой точки на 60-80%, но и минимизирует вероятность упущенных продаж из-за ситуаций «Out of Stock» (отсутствие товара на полке). Таким образом, переход к технологиям компьютерного зрения трансформирует мерчендайзинг из контролирующей функции в инструмент оперативного управления продажами, обеспечивая прозрачность и высокую точность мониторинга ритейл-среды.

Компания «Санта Импэкс» является одним из ведущих дистрибьюторов потребительских товаров в Республике Беларусь. Основным объектом контроля выкладки в данной работе выступает продукция, реализуемая через розничные торговые точки — магазины сетевой и несетевой розницы. Каждая товарная позиция обладает набором идентификационных характеристик: уникальный код SKU (Stock Keeping Unit), внешний вид упаковки, цветовая схема, форма и размер. Для формирования качественной обучающей выборки был проведен анализ морфологических признаков упаковок, включая их геометрию, тип материала и специфику цветовых решений бренда. Именно совокупность этих признаков позволяет системе компьютерного зрения однозначно идентифицировать товар

на фотографии торговой полки. Виды продукции «Санта Импэкс», подлежащие контролю выкладки, представлены на рисунке 1.1.



Рисунок 1.1 – Примеры продукции компании «Санта Импэкс» на торговых полках

Мерчендайзинг – это комплекс мероприятий, направленных на увеличение продаж непосредственно в торговой точке за счёт правильного размещения товаров, оформления полочного пространства и поддержания стандартов выкладки [1]. Мерчендайзер – специалист, осуществляющий регулярные визиты в торговые точки с целью проверки и корректировки выкладки товаров в соответствии с установленными стандартами компании.

Планограмма – это эталонная схема расположения товаров на торговом оборудовании, определяющая порядок размещения SKU, количество фейсингов и уровень полки [2]. Отклонение реальной выкладки от планограммы называется нарушением выкладки. Именно планограмма служит эталоном, с которым система «Digital Shelf» сравнивает результаты нейросетевого анализа фотографий.

Фейсинг – одна единица товара, обращённая лицевой стороной к покупателю. Количество фейсингов напрямую влияет на заметность товара на полке. Правильное число фейсингов каждой позиции закреплено в планограмме и является обязательным к соблюдению.

Доля полки (Share of Shelf, SOS) – ключевой показатель присутствия бренда в торговой точке. Он рассчитывается как отношение числа фейсингов конкретного бренда к общему числу фейсингов в данной категории по формуле (1.1):

$$SOS = \frac{F_{brand}}{F_{total}} \cdot 100\% \quad (1.1)$$

где SOS – доля полки бренда, %;

F_{brand} – количество фейсингов товаров бренда «Санта Импэкс»;

F_{total} – общее количество фейсингов в данной товарной категории.

OSA (On-Shelf Availability) – показатель наличия товара на полке. Определяет долю SKU из планограммы, фактически присутствующих в выкладке, и рассчитывается по формуле (1.2):

$$OSA = \frac{N_{present}}{N_{plan}} \cdot 100\% \quad (1.2)$$

где OSA – показатель наличия товара на полке, %;

$N_{present}$ – количество SKU, фактически присутствующих в выкладке;

N_{plan} – количество SKU, предусмотренных планограммой.

Таким образом, предметная область данной работы охватывает процессы мерчендайзинга продукции «Санта Импэкс», количественные показатели качества выкладки (SOS, OSA), а также методы автоматизированного контроля соответствия выкладки стандартам компании с применением Telegram-бота и нейронной сети.

1.2 Обследование процесса мерчендайзинга и выявление проблем

Текущий процесс контроля выкладки продукции «Санта Импэкс» в торговых точках осуществляется мерчендайзерами вручную. Сотрудник посещает торговую точку согласно маршрутному листу, визуально сверяет выкладку товаров с бумажной или электронной версией планограммы, фиксирует нарушения в отчётной форме и при необхо-

димости самостоятельно корректирует расположение товаров. Схема текущего процесса мерчендайзинга представлена на рисунке 1.2.

РУЧНОЙ ПРОЦЕСС КОНТРОЛЯ ВЫКЛАДКИ ТОВАРОВ

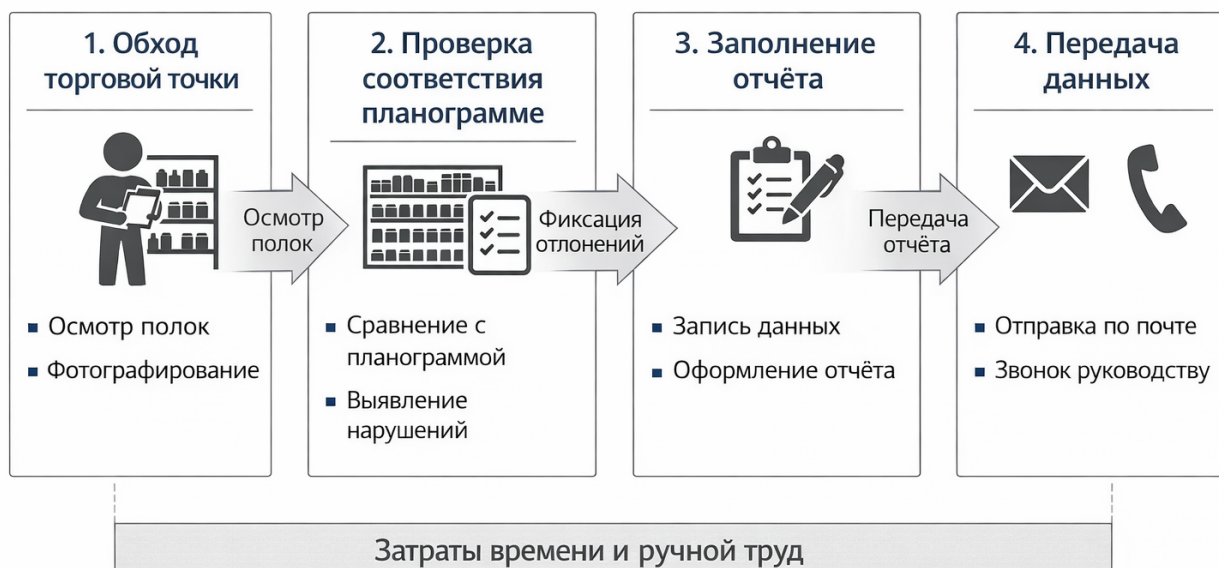


Рисунок 1.2 – Схема текущего (ручного) процесса контроля выкладки

Данный подход обладает рядом существенных недостатков. Во-первых, ручная проверка требует значительных временных затрат: в среднем мерчендайзер тратит от 15 до 30 минут на проверку одной торговой секции при ассортименте свыше 30 SKU. Во-вторых, результаты проверки зависят от субъективного восприятия специалиста и степени его внимательности, что неизбежно приводит к пропускам нарушений. В-третьих, отсутствие цифровой фиксации состояния выкладки не позволяет накапливать историческую аналитику для выявления систематических нарушений в конкретных торговых точках. В-четвёртых, ручное заполнение отчётных форм занимает от 20 до 35 % времени каждого визита, что снижает производительность труда мерчендайзера.

Для оценки эффективности работы мерчендайзера используется коэффициент выполнения задания (КВЗ), рассчитываемый по формуле (1.3):

$$KBZ = \frac{T_{completed}}{T_{total}} \cdot 100\% \quad (1.3)$$

где KBZ – коэффициент выполнения задания, %;

$T_{completed}$ – количество выполненных заданий за период;

T_{total} – общее количество запланированных заданий за период.

Работа мерчендайзера охватывает несколько ключевых задач в каждой торговой точке:

1. Проверка наличия всех SKU из планограммы.
2. Контроль количества фейсингов каждой позиции.

3. Проверка правильности зонирования и порядка расположения товаров.
4. Контроль наличия и корректности ценников.
5. Проверка соответствия позиций уровням полок согласно планограмме.
6. Фотофиксация текущего состояния торгового оборудования.
7. Заполнение отчёта о выявленных нарушениях и отправка супервайзеру.

Автоматизация данного процесса с применением системы «Digital Shelf» позволяет сократить время обработки одной фотографии до 5 секунд на сервере за счёт нейросетевого анализа изображений. Мерчендайзер лишь делает фотографию и отправляет её в Telegram-бот — система автоматически выполняет все остальные шаги. Схема автоматизированного процесса представлена на рисунке 1.3.



Рисунок 1.3 – Схема автоматизированного процесса контроля выкладки через систему «Digital Shelf»

Из сравнения схем видно, что автоматизированный процесс полностью исключает этапы ручного сравнения с планограммой и заполнения бумажного отчёта, заменяя их нейросетевым анализом фотографии и автоматической генерацией структурированного отчёта с визуальной разметкой нарушений прямо в интерфейсе Telegram-бота.

Таким образом, существующий ручной процесс контроля мерчендайзинга в компании «Санта Импэкс» характеризуется высокой трудоёмкостью, субъективностью оценки и отсутствием оперативной аналитики. Автоматизация данного процесса на основе технологий компьютерного зрения и Telegram-бота является актуальной и экономически обоснованной задачей.

1.3 Введение в компьютерное зрение, нейронные сети и машинное обучение

Опишем средства, которые будут использоваться для автоматического анализа фотографий торговых полок, а также введём основные понятия, необходимые для понимания работы нейросетевых моделей компьютерного зрения.

Цифровое изображение представляет собой двумерный массив пикселей. Каждый пиксель в цветовой модели RGB характеризуется тремя значениями интенсивности цветных каналов. Таким образом, изображение размером $H \times W$ пикселей представляется тензором по формуле (1.4):

$$I \in \mathbb{R}^{H \times W \times 3} \quad (1.4)$$

где H – высота изображения в пикселях;

W – ширина изображения в пикселях;

3 – количество цветных каналов (R, G, B).

Задача детекции объектов на изображении сводится к нахождению ограничивающих прямоугольников (bounding boxes) для каждой товарной позиции. Ограничивающий прямоугольник задаётся вектором по формуле (1.5):

$$\hat{b} = (x_c, y_c, w, h, c, p_c) \quad (1.5)$$

где x_c, y_c – координаты центра ограничивающего прямоугольника;

w, h – ширина и высота прямоугольника;

c – предсказанный класс объекта (SKU);

p_c – степень уверенности модели в правильности предсказания.

Рассмотрим нейронную сеть как некоторую математическую модель и её программную реализацию. Рассматривая нейронную сеть как «чёрный ящик», изображённый на рисунке 1.4, мы понимаем, что на вход подаётся фотография торговой полки, сделанная мерчендайзером через Telegram-бот, а на выходе формируется набор предсказанных ограничивающих прямоугольников с идентификаторами SKU и оценками уверенности.

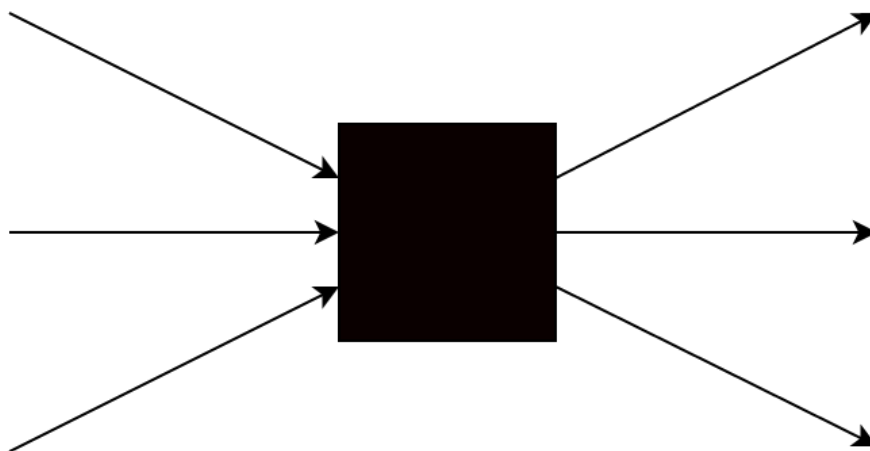


Рисунок 1.4 – Представление нейронной сети как «чёрный ящик»

Нейронная сеть, рассматриваемая как «белый ящик», состоит из нейронов, со-

бранных в упорядоченные слои. Входной слой принимает тензор изображения, скрытые слои последовательно извлекают признаки возрастающего уровня абстракции, а выходной слой формирует предсказания координат и классов объектов. Пример архитектуры нейронной сети с двумя скрытыми слоями представлен на рисунке 1.5.

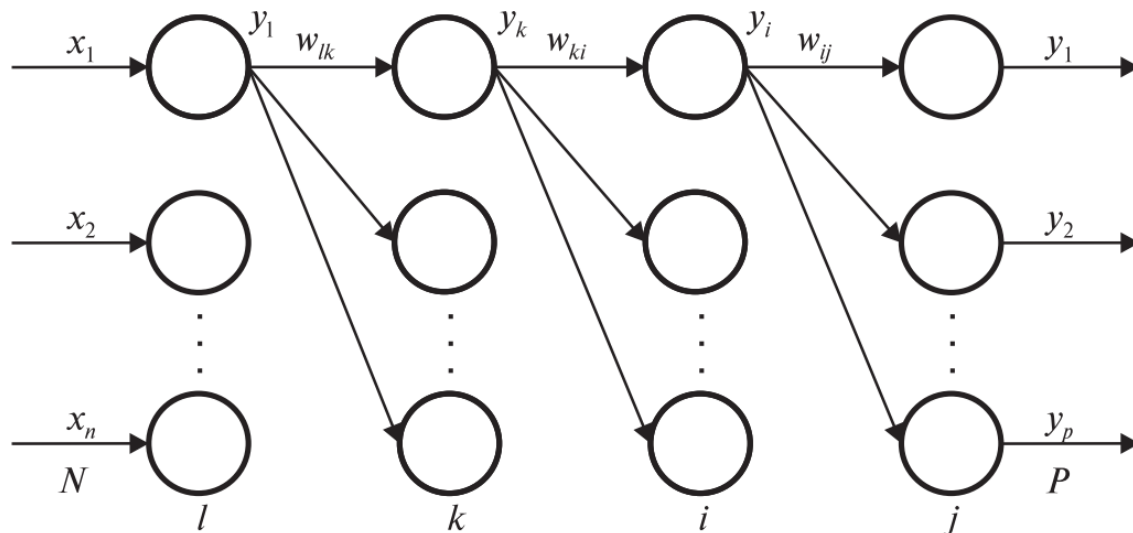


Рисунок 1.5 – Пример архитектуры нейронной сети с двумя скрытыми слоями

Каждый нейрон состоит из сумматора и функции активации. Сумматор вычисляет взвешенную сумму входных значений по формуле (1.6):

$$S = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b \quad (1.6)$$

где S – взвешенная сумма;

x_n – n -тое входное значение нейрона;

w_n – n -тый весовой коэффициент;

b – коэффициент смещения.

Выходное значение нейрона рассчитывается по формуле (1.7):

$$y = F(S) \quad (1.7)$$

где $F(\dots)$ – функция активации нейрона;

y – выходное значение нейронного элемента.

Структура нейронного элемента показана на рисунке 1.6.

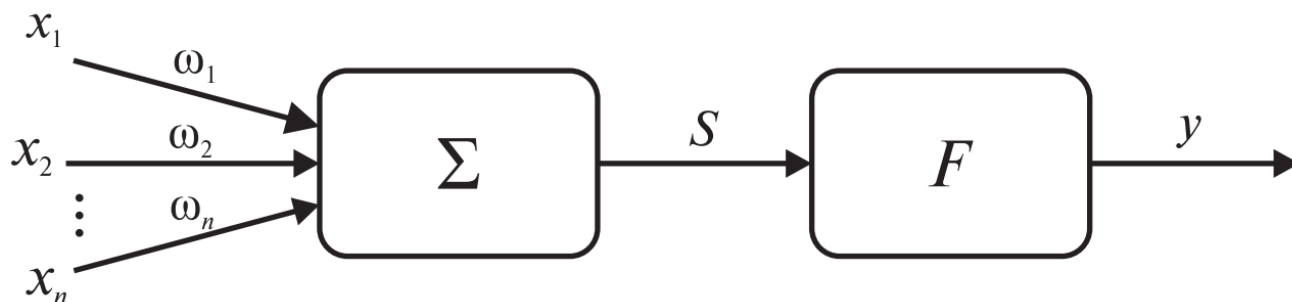


Рисунок 1.6 – Структура нейронного элемента

Для задачи детекции товарных позиций на полках применяются свёрточные ней-

ронные сети (CNN). Ключевой операцией в CNN является свёртка, позволяющая извлекать локальные визуальные признаки – края упаковок, характерные цветовые паттерны, логотипы брендов – независимо от их положения на изображении. Операция свёртки для одного канала записывается по формуле (1.8):

$$S(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (1.8)$$

где I – входной тензор признаков;
 K – ядро свёртки (фильтр);
 S – результирующая карта признаков.

Качество детектора оценивается метрикой mAP (mean Average Precision), которая вычисляется через метрику IoU (Intersection over Union) – меру перекрытия предсказанного и эталонного прямоугольников. IoU рассчитывается по формуле (1.9):

$$IoU = \frac{|B_{pred} \cap B_{gt}|}{|B_{pred} \cup B_{gt}|} \quad (1.9)$$

где B_{pred} – предсказанный ограничивающий прямоугольник;
 B_{gt} – эталонный ограничивающий прямоугольник (ground truth).

Функция потерь при обучении детектора включает три составляющих: ошибку классификации, ошибку локализации и ошибку объектности. Суммарная функция потерь записывается по формуле (1.10):

$$L = \lambda_{cls} \cdot L_{cls} + \lambda_{loc} \cdot L_{loc} + \lambda_{obj} \cdot L_{obj} \quad (1.10)$$

где L_{cls} – составляющая потери по классификации;
 L_{loc} – составляющая потери по локализации;
 L_{obj} – составляющая потери по объектности;
 λ_{cls} , λ_{loc} , λ_{obj} – весовые коэффициенты.

Обновление весовых коэффициентов сети в процессе обучения выполняется методом градиентного спуска по формуле (1.11):

$$w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w_t} \quad (1.11)$$

где w_t – значения весов на шаге t ;
 α – скорость обучения (learning rate);
 $\frac{\partial L}{\partial w_t}$ – градиент функции потерь по весам.

В задаче детекции товаров «Санта Импэкс» применяется подход переноса обучения (transfer learning): нейронная сеть предварительно обучается на крупных открытых датасетах (COCO, ImageNet), а затем дообучается на специфическом датасете из фотографий продукции компании. Принципиальная схема работы детектора YOLOv8 при обработке фотографии, полученной от мерчендайзера через Telegram-бот, представлена на рисунке 1.7.

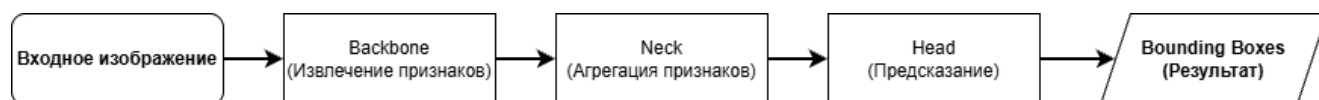


Рисунок 1.7 – Принципиальная схема работы детектора YOLOv8 в системе «Digital Shelf»

После получения изображения бот передает его на сервер, где выполняется пред-

варительная обработка данных, включая изменение размера, нормализацию и, при необходимости, подавление шума. Затем изображение поступает на вход нейронной сети, которая за один проход (single-shot) определяет наличие объектов и их координаты на изображении. Модель разбивает изображение на сетку и для каждой ячейки предсказывает вероятности наличия объектов и ограничивающие рамки (bounding boxes), что обеспечивает высокую скорость обработки.

Далее выполняется этап постобработки, включающий применение алгоритма подавления немаксимумов (Non-Maximum Suppression) для устранения дублирующихся предсказаний. После этого формируется итоговый список обнаруженных товаров с указанием их классов и координат. Полученные результаты могут использоваться для анализа выкладки товаров, контроля наличия продукции и проверки соответствия планограмме. Итоговая информация передается пользователю через Telegram-бот в виде аннотированного изображения или структурированного отчета. Дополнительно система может сохранять результаты обработки в базе данных для последующего анализа и формирования статистики.

Таким образом, задача автоматического анализа фотографий торговых полок формулируется как задача обучения с учителем в области компьютерного зрения, решаемая с применением свёрточных нейронных сетей архитектуры YOLOv8.

1.4 Требования к системе «Digital Shelf» и постановка задачи

Система «Digital Shelf» должна обеспечивать автоматическую проверку выкладки продукции «Санта Импэкс» в торговых точках. Для этого система принимает фотографию торговой полки, отправленную мерчендайзером через Telegram-бота, производит детальный нейросетевой анализ изображения и возвращает пользователю аннотированный результат непосредственно в чат. Структурная схема системы «Digital Shelf» представлена на рисунке 1.8.

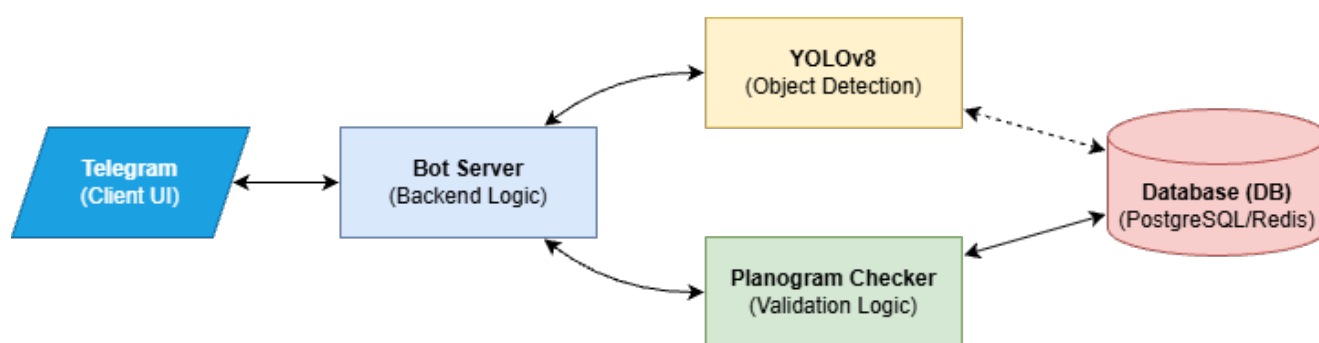


Рисунок 1.8 – Структурная схема системы «Digital Shelf»

К системе предъявляются следующие функциональные требования:

1. Приём фотографии торговой полки от мерчендайзера через интерфейс Telegram-бота.
2. Детекция и классификация всех товарных позиций «Санта Импэкс», попавших в кадр, с точностью не менее $mAP_{0,5} \geq 0,80$.
3. Сравнение результатов распознавания с эталонной планограммой торговой точ-

ки и формирование перечня нарушений.

4. Визуальная аннотация исходного изображения: корректно расположенные товары выделяются зелёными рамками, нарушения – красными.

5. Автоматическое формирование структурированного отчёта с расчётом показателей OSA и SOS и отправка его обратно в Telegram.

6. Время обработки одного изображения – не более 10 секунд с момента получения фотографии сервером.

7. Сохранение обработанного изображения и отчёта в базе данных с привязкой к торговой точке, дате и идентификатору мерчендайзера.

К системе также предъявляются нефункциональные требования. Система должна быть устойчива к вариативности входных изображений: различным условиям освещения, бликам, частичному перекрытию товаров и различным углам съёмки. Интерфейс Telegram-бота должен быть понятен сотруднику без специальной технической подготовки и не требовать установки дополнительного программного обеспечения на мобильное устройство. Серверная часть должна обеспечивать одновременную обработку запросов не менее чем от 50 пользователей без деградации производительности.

Задачей данного проекта является разработка системы «Digital Shelf» – программного обеспечения автоматизированного контроля выкладки товаров «Санта Импэкс» в торговых точках, реализованного в виде Telegram-бота с интегрированным нейросетевым модулем на основе YOLOv8, модулем сравнения с планограммой и модулем автоматического формирования отчётов.

Для выполнения поставленной задачи необходимо: собрать и разметить датасет фотографий торговых полок с продукцией «Санта Импэкс»; выбрать и дообучить нейросетевую модель YOLOv8 на специфическом датасете; разработать Telegram-бот на базе библиотеки aiogram; реализовать алгоритм сопоставления результатов детекции с планограммой; обеспечить хранение результатов в реляционной базе данных; провести тестирование и оценку качества системы по метрикам mAP, Precision и Recall.

2. АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ

В данной главе проводится анализ существующих программных решений в области автоматизированного контроля выкладки товаров в розничной торговле с применением компьютерного зрения, а также обосновывается выбор технологического стека для системы «Retail Vision».

2.1 Обзор существующих решений и обоснование разработки

Задача автоматического распознавания товаров на полках магазинов активно исследуется с середины 2010-х годов. К наиболее известным коммерческим платформам относятся **Trax Retail**, **Vispera** и **Planorama**. Эти системы предоставляют функциональность распознавания SKU, анализа доли полки и автоматического формирования отчётов [10, 11]. Однако все они являются закрытыми платформами с высокой стоимостью лицензирования, не допускают обучения на пользовательских датасетах и не предоставляют интерфейса на основе Telegram.

Сравнительный анализ рассмотренных систем и разрабатываемого решения представлен в таблице 2.1.

Таблица 2.1 – Сравнительный анализ существующих решений

Характеристика	Trax	Vispera	Planorama	Retail Vision
Интерфейс Telegram	Нет	Нет	Нет	Да
Собственный датасет	Нет	Нет	Нет	Да
Открытый исходный код	Нет	Нет	Нет	Да
Интеграция с планограммой	Да	Да	Да	Да
Поддержка рынка РБ	Нет	Нет	Нет	Да
Стоимость	Высокая	Высокая	Средняя	Низкая

Таким образом, ни один из аналогов не удовлетворяет комплексу требований компании «Санта Импэкс», что подтверждает целесообразность разработки собственной системы.

2.2 Архитектура YOLOv8 и её применимость к задаче

В качестве нейросетевой основы системы выбрана модель **YOLOv8** — однопроходный детектор объектов, разработанный компанией Ultralytics в 2023 году [2]. YOLOv8 выполняет одновременно классификацию и локализацию объектов за один прямой проход через сеть. По сравнению с предшественниками архитектура предлагает улучшенный backbone CSPDarknet с блоками C2f и безанкорный (anchor-free) механизм детекции, что повышает точность локализации объектов разного размера. Архитектура YOLOv8 пред-

ставлена на рисунке 2.1.

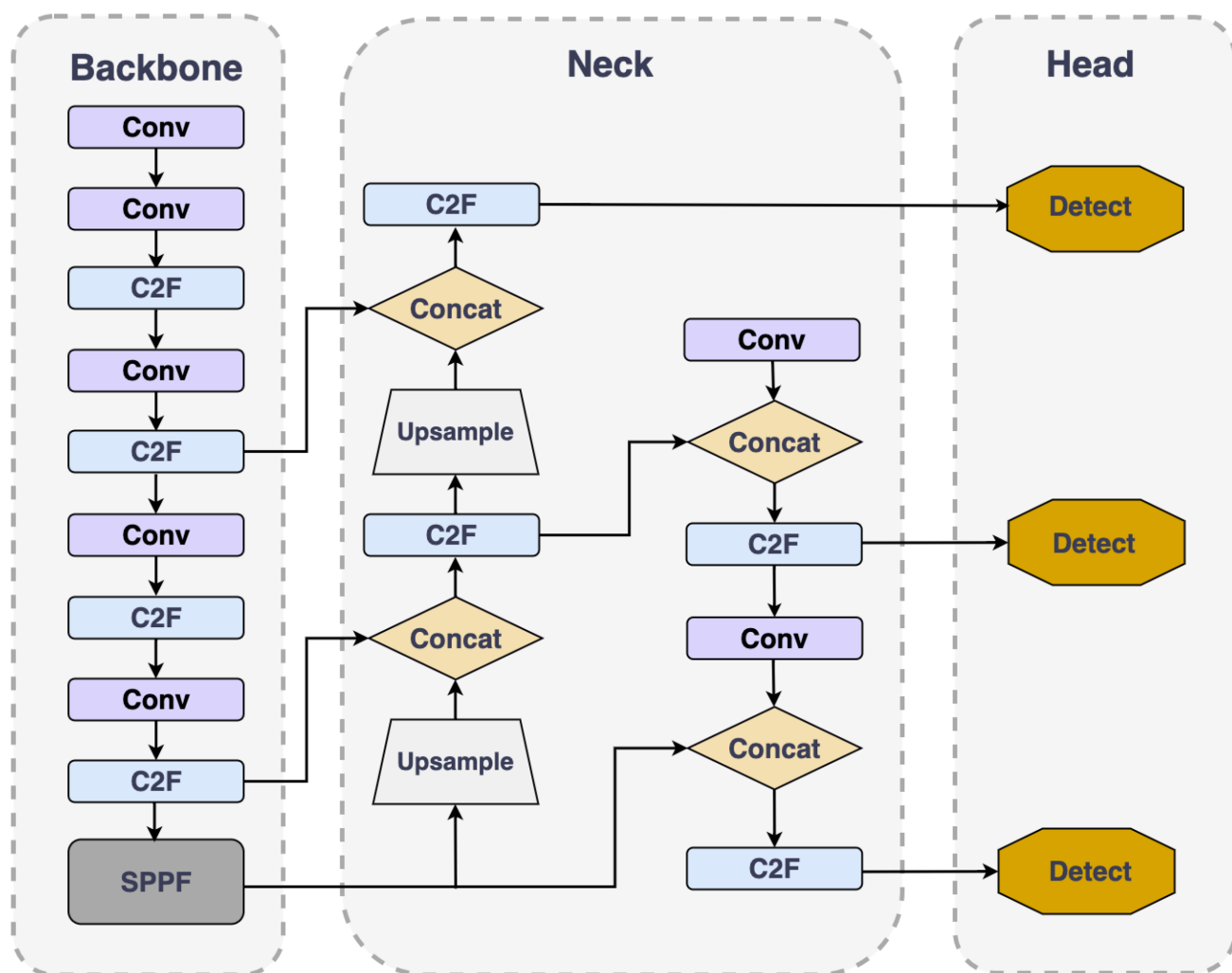


Рисунок 2.1 – Архитектура нейронной сети YOLOv8

Общая функция потерь при обучении YOLOv8 записывается по формуле (2.1):

$$\mathcal{L} = \lambda_{\text{box}} \cdot \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \cdot \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \cdot \mathcal{L}_{\text{dfl}} \quad (2.1)$$

где $\mathcal{L}_{\text{box}}$ – потеря регрессии ограничивающего прямоугольника;

$\mathcal{L}_{\text{cls}}$ – потеря классификации (Binary Cross-Entropy);

$\mathcal{L}_{\text{dfl}}$ – Distribution Focal Loss для уточнения координат;

λ_{box} , λ_{cls} , λ_{dfl} – весовые коэффициенты компонент.

Качество детекции оценивается метрикой mAP, рассчитываемой по формуле (2.2):

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (2.2)$$

где N – количество классов объектов (SKU);

AP_i – средняя точность для i -го класса — площадь под кривой точность-полнота.

2.3 Архитектура Telegram-бота на базе aiogram

Telegram-бот «Retail Vision» разработан с использованием фреймворка **aiogram** — асинхронной Python-библиотеки для работы с Telegram Bot API [15]. Выбор aiogram обусловлен его асинхронной архитектурой на основе `asyncio`, позволяющей обслуживать множество одновременных запросов без блокировки потока выполнения.

Поток данных в системе выглядит следующим образом: мерчендайзер отправляет фотографию в бот, обработчик принимает изображение и передаёт его в сервисный слой, где выполняется предобработка (ресайз до 640×640 пикселей) и инференс YOLOv8. Результаты детекции сравниваются с планограммой, после чего аннотированный снимок и текстовый отчёт возвращаются в Telegram. Там пользователь (мерчендайзер) может увидеть результат обработки и анализа системы. Суммарное время обработки запроса рассчитывается по формуле (2.3):

$$t_{\text{total}} = t_{\text{recv}} + t_{\text{pre}} + t_{\text{forward}} + t_{\text{nms}} + t_{\text{send}} \quad (2.3)$$

где t_{recv} — время получения изображения от Telegram API;

t_{pre} — время предобработки (ресайз, нормализация);

t_{forward} — время инференса нейронной сети YOLOv8;

t_{nms} — время выполнения Non-Maximum Suppression;

t_{send} — время отправки результата в Telegram.

Целевое суммарное время обработки не должно превышать $t_{\text{total}} \leq 10$ с. Архитектура системы «Retail Vision» представлена на рисунке 2.2.

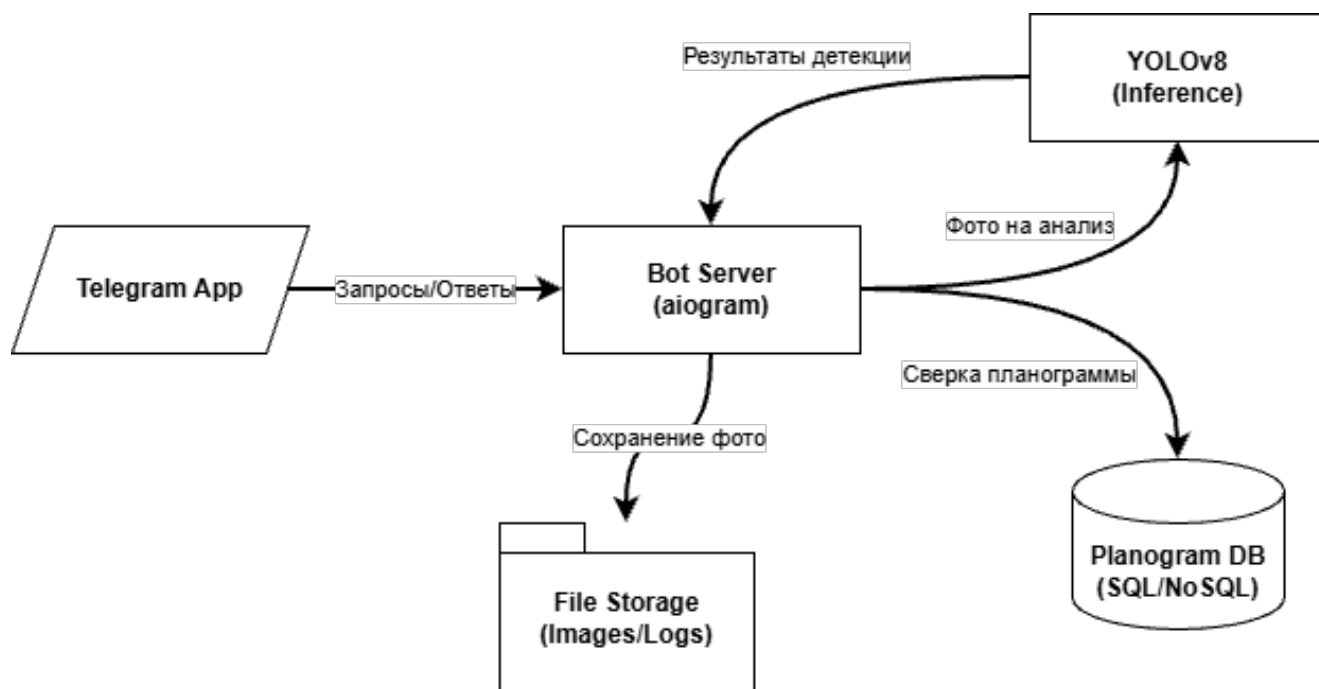


Рисунок 2.2 – Архитектура системы «Retail Vision» и взаимодействие компонентов

Представленная модульная структура обеспечивает гибкость системы, позволяя проводить обновление весов нейросетевой модели или модификацию алгоритмов сверки без необходимости изменения базовой логики Telegram-бота. Такое разделение ответственности между компонентами гарантирует высокую отказоустойчивость сервиса и упро-

щает процесс его горизонтального масштабирования при увеличении интенсивности входящего потока данных от пользователей.

2.4 Оценка соответствия выкладки планограмме

После нейросетевой детекции товаров модуль проверки планограммы осуществляет сопоставление множества обнаруженных SKU с эталонным составом продукции для конкретной торговой точки, используя алгоритм анализа пространственного расположения ограничивающих рамок (bounding boxes) относительно виртуальной сетки полок. В процессе этого анализа система учитывает не только факт присутствия товара, но и его точные координаты, что позволяет идентифицировать нарушения последовательности выкладки или ошибки товарного соседства. Позиции, присутствующие в требуемом объеме и на своих местах, визуальнo помечаются зелеными рамками, в то время как отсутствующие или некорректно размещенные товары выделяются красным цветом для привлечения внимания мерчендайзера. На основе полученных данных система автоматически рассчитывает ключевые показатели эффективности, такие как доступность товара на полке (OSA) и доля полки (SOS), формируя итоговый аналитический отчет, включающий аннотированное изображение и детальный перечень выявленных нарушений. Пример работы данного модуля и формирования ответа в интерфейсе Telegram-бота представлен на рисунке 2.3.

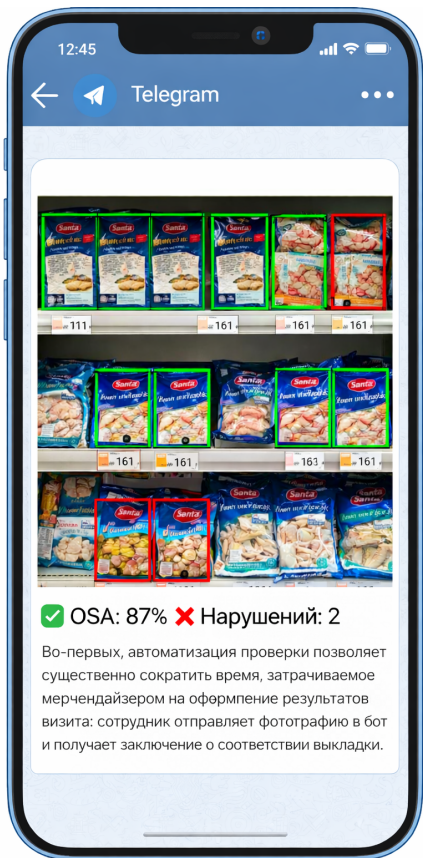


Рисунок 2.3 – Пример ответа системы «Retail Vision» в интерфейсе Telegram-бота

Расчет ключевых бизнес-метрик является важнейшим аналитическим этапом работы системы. Показатель доступности товара на полке (On-Shelf Availability, OSA) вы-

числяется как отношение фактически распознанных позиций к эталонному количеству, регламентированному в планеграмме. В свою очередь, доля полки (Share of Shelf, SOS) определяется как процентное соотношение фейсингов продукции «Санта Импэкс» к общему числу идентифицированных товаров на стеллаже. Автоматизация вычисления данных показателей нивелирует влияние субъективного фактора и позволяет менеджменту компании получать прозрачную аналитику в режиме реального времени.

Для обеспечения отказоустойчивости архитектуры «Retail Vision» при промышленной эксплуатации применяется подход логического разделения компонентов. Изоляция процессов Telegram-бота, тяжелых вычислительных задач инференса YOLOv8 и механизмов работы с базой данных позволяет эффективно распределять ресурсы сервера. В условиях пиковых нагрузок, возникающих при одновременной отправке фотографий десятками мерчендайзеров, асинхронная природа фреймворка aiogram гарантирует стабильную постановку запросов в очередь, предотвращая потерю данных и обеспечивая соблюдение установленных временных ограничений на обработку.

Таким образом, проведенный анализ подтверждает обоснованность разработки системы «Retail Vision». Использование YOLOv8 обеспечивает необходимую точность и скорость инференса, применение aiogram позволяет построить масштабируемый асинхронный бот, а формирование собственного датасета гарантирует высокое качество распознавания продукции «Санта Импэкс».

3. СБОР И ПОДГОТОВКА ДАТАСЕТА ПРОДУКЦИИ САНТА ИМПЭКС

3.1 Анализ и структура собираемых данных

Ключевым условием успешного обучения модели детекции объектов является наличие качественного и представительного датасета. Поскольку специализированных открытых датасетов с изображениями продукции компании Санта Импэкс не существует, датасет для обучения модели YOLOv8n собирается самостоятельно в торговых точках, реализующих мороженое данной компании.

Датасет включает изображения продукции четырёх торговых марок: **ТОП**, **Soletto**, **ЮККИ** и **ТОП ЛЕД**, охватывающих различные форматы упаковки: рожки, стаканчики, эскимо, контейнеры и пакеты. Полный перечень классов SKU представлен в таблице 3.1.

Таблица 3.1 – Классы датасета продукции Санта Импэкс.

ID	Бренд	Серия	Вкус / Граммовка
0	ТОП	Рожок	Клубника со сливками, 75 г
1	ТОП	Рожок	Фисташка, 75 г
2	ТОП	Стаканчик	Шоколад / Ваниль, 65 г
3	ТОП	Эскимо	Карамель / Арахис, 70 г
4	ТОП	ТРИО (пакет)	Клубника-Ваниль-Шоколад, 400 г
5	Soletto	Рожок Classico	Лаванда-Черника, 75 г
6	Soletto	Рожок Classico	Фисташка-Марципан, 75 г
7	Soletto	Рожок Classico	Гранат-Лимон, 75 г
8	Soletto	Gourmet (стакан)	Кедровый орех-Гауда, 190 г
9	Soletto	Gourmet (стакан)	Шоколад-Брауни, 190 г
10	ЮККИ	Пломбир (стакан)	На сливках (ваниль), 65 г
11	ЮККИ	Пломбир (эскимо)	В шоколадной глазури, 70 г
12	ЮККИ	Контейнер	Муравейник (с маком/печеньем), 450 г
13	ЮККИ	Семейное (пакет)	Ваниль / Шоколад, 400 г
14	ТОП ЛЕД	Фруктовый лёд	Малина-Лимон / Арбуз, 70 г

Итого датасет включает $N_{cls} = 15$ классов объектов. С точки зрения инженерии машинного обучения необходимо оценить достаточность данных для обучения. Для обеспечения статистической репрезентативности каждого класса при обучении YOLOv8 минимально необходимое количество аннотированных объектов на класс составляет $n_{obj} = 300$. Тогда минимальный объём датасета определяется по формуле (3.1):

$$N_{min} = N_{cls} \times n_{obj} = 15 \times 300 = 4500 \text{ объектов} \quad (3.1)$$

Изображения для датасета собираются непосредственно в торговых точках при реальных условиях эксплуатации. Для обеспечения достаточного разнообразия обучающей выборки съёмка производится в различных условиях: при разном освещении (дневное освещение торгового зала, искусственный свет холодильного оборудования), под разными

ми углами (фронтальный, под углом 15–30° к плоскости полки), при различной степени заполненности полок и в разных торговых точках. Такой подход позволяет повысить обобщающую способность обученной модели.

С точки зрения качества данных собираемый датасет обладает следующими характеристиками. Данные являются информативными, поскольку отражают реальные условия работы мерчендайзера. Данные соответствуют реальным входным данным системы: модель будет обучаться на изображениях, аналогичных тем, которые будут поступать от камеры мобильного устройства в процессе эксплуатации. Данные охватывают все классы продукции, представленные в ассортиментной матрице Санта Импэкс.

Для дальнейшего описания процесса сбора и анализа данных необходима их визуализация. Пример собранных изображений различных SKU продукции Санта Импэкс на полках холодильного оборудования представлен на рисунке 3.1.



Рисунок 3.1 – Примеры изображений из собранного датасета продукции Санта Импэкс.

Анализ собранных изображений позволяет выявить ряд характерных сложностей, которые необходимо учесть при подготовке датасета и настройке модели. Во-первых, ряд SKU имеют схожую цветовую гамму упаковки (например, рожок ТОП «Клубника со сливками» и рожок Soletto «Гранат-Лимон» содержат красно-розовые оттенки), что потенциально ведёт к смешению классов. Во-вторых, упаковки в формате пакетов (ТОП ТРИО, ЮККИ Семейное) существенно отличаются по размеру от штучных позиций, что влияет

на выбор масштаба детекции. В-третьих, часть позиций может быть частично перекрыта другими упаковками или ценниками, что создаёт сложные условия для детекции.

Также важно отметить отсутствие данных с аномальными условиями съёмки: размытые изображения, слишком тёмные или засвеченные кадры, изображения с нехарактерными углами съёмки исключаются из датасета на этапе первичной фильтрации. Это позволяет обеспечить высокое качество обучающей выборки.

3.2 Разметка данных и формирование датасета

После сбора изображений необходимо выполнить их аннотирование (разметку) — процесс ручной разметки объектов на каждом изображении с указанием координат ограничивающих прямоугольников (bounding boxes) и соответствующих меток классов. Разметка выполняется с использованием платформы **Roboflow**, предоставляющей удобный веб-интерфейс для аннотирования изображений и управления датасетом.

Формат YOLO использует нормализованные координаты, что обеспечивает независимость разметки от разрешения исходного изображения. Каждая строка текстового файла аннотации включает в себя индекс класса и четыре вещественных числа: координаты центра ограничивающего прямоугольника (x, y), а также его ширину (w) и высоту (h). Значения координат варьируются в диапазоне от 0 до 1, где начало отсчета находится в верхнем левом углу изображения. Такая система представления данных позволяет эффективно подавать изображения разных размеров на вход нейронной сети без необходимости пересчета координат вручную.

В процессе разметки особое внимание уделяется точности позиционирования рамок и полноте охвата объектов. Для проекта «Retail Vision» критически важным является исключение фрагментарных перекрытий, когда один объект может быть ошибочно принят за несколько разных. Использование инструментов Roboflow позволяет автоматизировать часть рутинных операций, например, через применение функций интеллектуального выделения контуров или копирования разметки для схожих ракурсов, что существенно ускоряет подготовку массива данных объемом в несколько сотен или тысяч экземпляров.

Завершающим этапом подготовки является верификация аннотаций, направленная на выявление и исправление ошибок человеческого фактора: пропущенных объектов, неверно назначенных классов или избыточно широких границ рамок. Качественно размеченный датасет выступает фундаментом для обучения модели, так как наличие шума в обучающей выборке напрямую коррелирует с увеличением функции потерь и снижением метрик средней точности (mAP) при последующем тестировании системы.

Аннотации хранятся в стандартном для YOLO текстовом формате. Каждому изображению соответствует текстовый файл с расширением `.txt`, содержащий одну строку на каждый аннотированный объект в следующем виде:

```
<class_id> <x_c> <y_c> <w> <h>
```

где $class_id$ – целочисленный идентификатор класса объекта (от 0 до 14);
 x_c , y_c – нормализованные координаты центра ограничивающего прямоугольника относительно ширины и высоты изображения соответственно;
 w , h – нормализованные ширина и высота ограничивающего прямоугольника.
 Нормализация координат производится по формулам (3.2):

$$x_c = \frac{x_{left} + x_{right}}{2 \cdot W_{img}}, \quad y_c = \frac{y_{top} + y_{bottom}}{2 \cdot H_{img}} \quad (3.2)$$

где x_{left} , x_{right} – пиксельные координаты левой и правой границ объекта;
 y_{top} , y_{bottom} – пиксельные координаты верхней и нижней границ объекта;
 W_{img} , H_{img} – ширина и высота изображения в пикселях.

Все значения нормализованных координат принадлежат диапазону $[0; 1]$, что обеспечивает независимость формата аннотаций от разрешения изображений. Пример размеченного изображения с ограничивающими прямоугольниками и метками классов показан на рисунке 3.2.

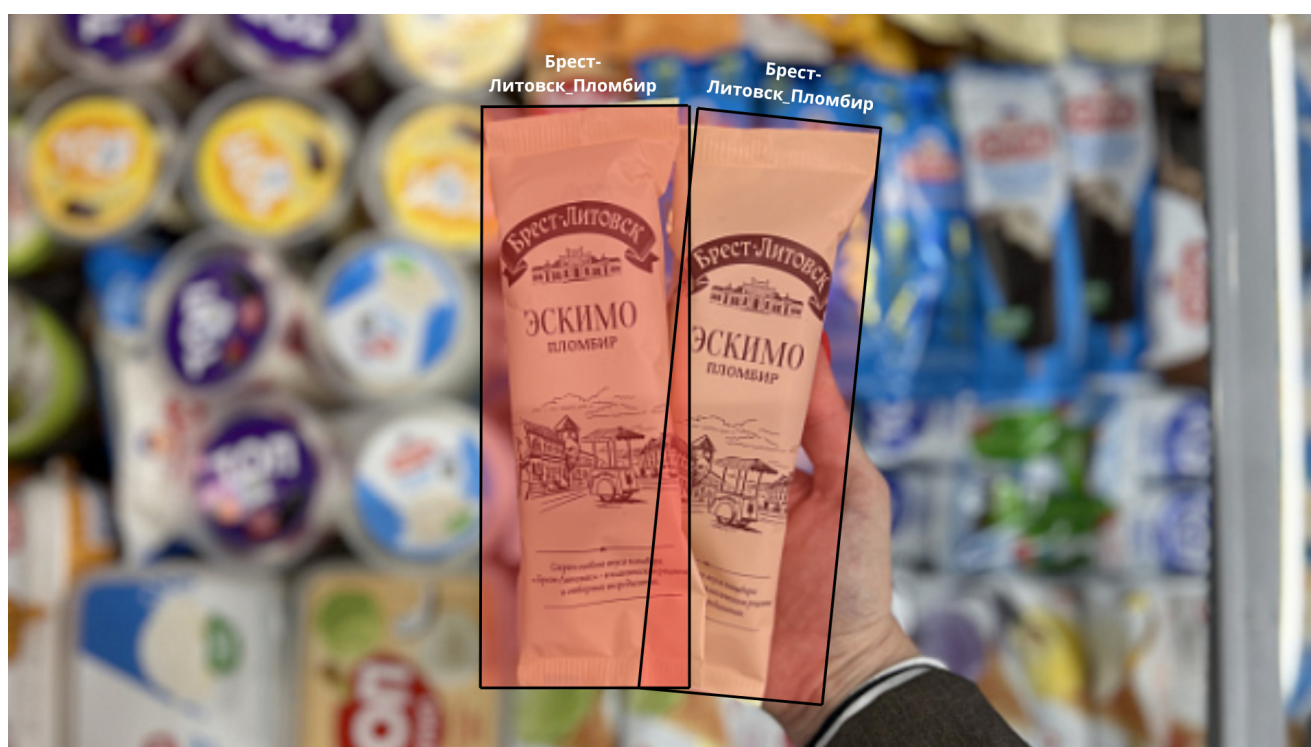


Рисунок 3.2 – Пример аннотированного изображения продукции Санта Импэкс с ограничивающими прямоугольниками.

Итоговый датасет разбивается на три подмножества в соответствии с требованиями инженерии машинного обучения. **Обучающая выборка** используется непосредственно для обновления весовых коэффициентов модели в процессе обучения. **Валидационная выборка** служит для оценки качества модели после каждой эпохи обучения и подбора гиперпараметров. **Тестовая выборка** используется для финальной оценки обученной модели на данных, не участвовавших ни в обучении, ни в валидации. Соотношение выборок составляет 70% / 15% / 15% соответственно.

Структура директорий датасета в формате Ultralytics YOLOv8 организована следующим образом:

dataset/ images/train/, images/val/, images/test/

dataset/ labels/train/, labels/val/, labels/test/

Конфигурационный файл датасета `data.yaml` содержит пути к директориям, количество классов и их имена. Этот файл передаётся модели YOLOv8 при запуске обучения.

Для обеспечения равномерного распределения классов в выборках используется стратифицированное разбиение датасета. Распределение количества аннотированных объектов по классам показано на рисунке 3.3.

Анализ представленной ниже гистограммы позволяет оценить степень дисбаланса классов, который является характерной особенностью прикладных задач компьютерного зрения в ритейле. Неравномерность распределения объектов обусловлена естественными различиями в представленности конкретных товарных позиций на торговых полках, а также вариативностью спроса на отдельные виды продукции. Визуализация данной проблемы непосредственно в условиях торговой точки представлена на рисунке 3.3.



Рисунок 3.3 – Визуализация дисбаланса классов на примере выкладки продукции: доминирование мажоритарного SKU над миноритарными.

На изображении отчетливо видно, что мажоритарные классы (выделены красным цветовым тоном) занимают фэйсинг, в несколько раз превосходящий площадь, отведенную под редкие SKU (выделены синим). Учет данной специфики на этапе подготовки данных критически важен для предотвращения смещения модели в сторону доминирующих позиций. В противном случае, нейронная сеть будет демонстрировать высокую точность на частых объектах, игнорируя или ошибочно классифицируя редкие, но критически важные для аналитики позиции, что приведет к существенному снижению метрик mAP при тестировании системы. Для минимизации подобных рисков в работе применяется комплексный подход, включающий не только стратификацию выборки, но и использование специализированных функций потерь, таких как Focal Loss, которые позволяют динамично

чески перевзвешивать вклад каждого примера в процесс обучения.

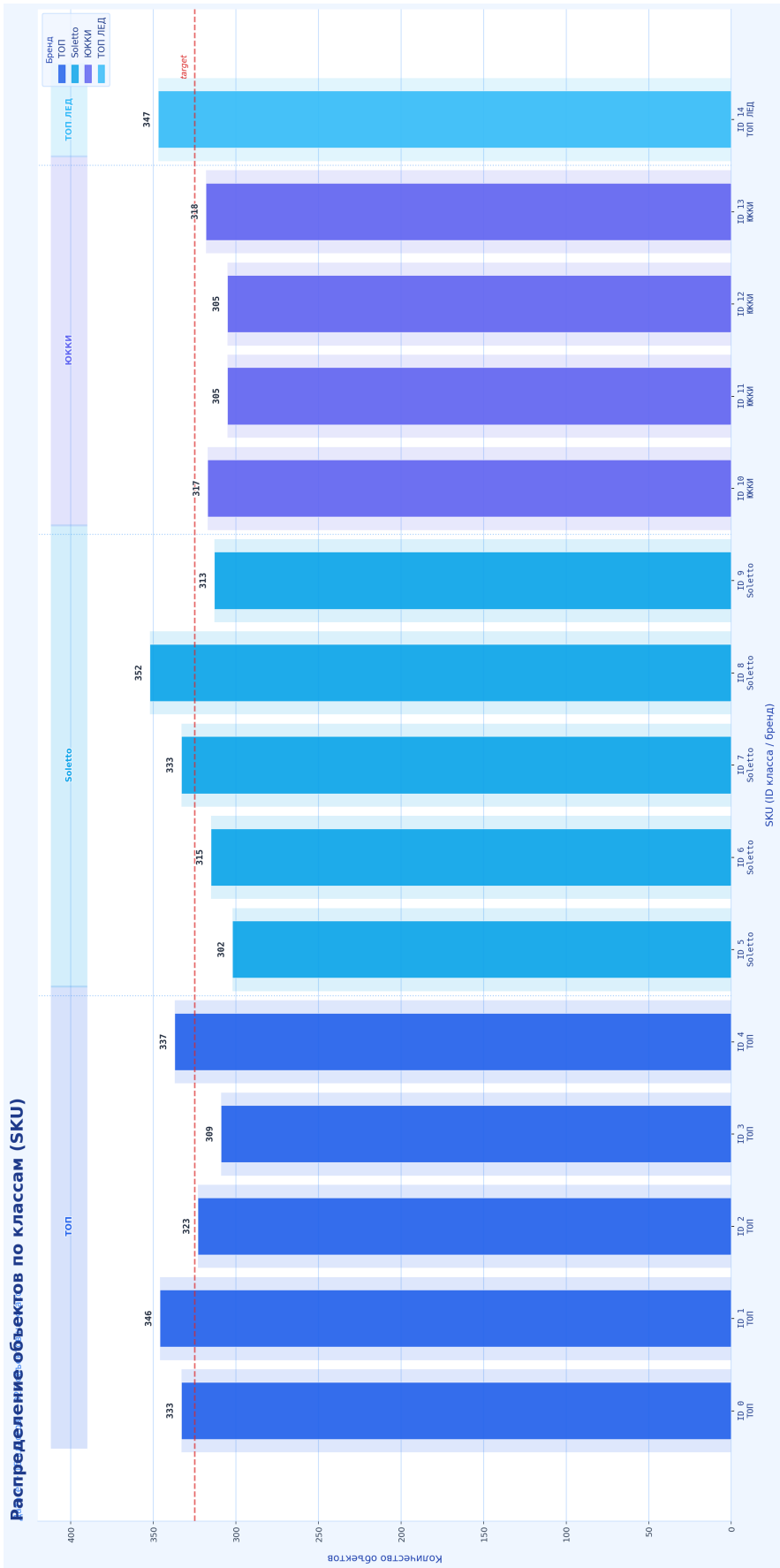


Рисунок 3.4 – Распределение количества объектов по классам в датасете.

3.3 Обработка данных и аугментация

После первичного сбора и разметки изображений необходимо выполнить их обработку и подготовку к обучению модели. Основными задачами данного этапа являются нормализация изображений, их масштабирование до входного размера модели и аугментация данных с целью расширения обучающей выборки и повышения обобщающей способности модели.

Первым шагом предобработки является масштабирование изображений до стандартного входного размера YOLOv8 640×640 пикселей. Поскольку исходные изображения, снятые камерой смартфона, имеют различные разрешения и соотношения сторон, применяется метод **letterbox-масштабирования**: изображение пропорционально уменьшается до тех пор, пока его большая сторона не достигнет 640 пикселей, а оставшееся пространство заполняется серыми полосами. Это исключает искажение пропорций объектов. Пересчёт координат аннотаций при letterbox-масштабировании выполняется по формуле (3.3):

$$s = \frac{imgsz}{\max(W_{img}, H_{img})} \quad (3.3)$$

где s – коэффициент масштабирования;

$imgsz = 640$ – целевой размер входного изображения.

Нормализация значений пикселей выполняется делением на 255 для приведения диапазона значений из $[0; 255]$ к диапазону $[0; 1]$ по формуле (3.4):

$$\hat{p} = \frac{p}{255} \quad (3.4)$$

где $p \in [0; 255]$ – исходное значение пикселя;

$\hat{p} \in [0; 1]$ – нормализованное значение пикселя.

Аугментация данных является ключевым инструментом расширения обучающей выборки и снижения риска переобучения модели. В пайплайне обучения YOLOv8 применяются следующие виды аугментации.

HSV-аугментация — случайное изменение тона (Hue), насыщенности (Saturation) и яркости (Value) изображения в заданных диапазонах. Это моделирует различные условия освещения в торговых точках. Параметры: $\Delta H \in [-0,015; 0,015]$, $\Delta S \in [0,7; 1,3]$, $\Delta V \in [0,4; 1,6]$.

Горизонтальное отражение (horizontal flip) — симметричное отражение изображения относительно вертикальной оси с вероятностью $p = 0,5$. При этом координаты аннотаций пересчитываются по формуле (3.5):

$$x'_c = 1 - x_c \quad (3.5)$$

Mosaic-аугментация — объединение четырёх случайно выбранных изображений обучающей выборки в одно изображение 640×640 пикселей с произвольной точкой объединения. Данный вид аугментации позволяет модели обучаться на частично видимых объектах и объектах, расположенных у краёв изображения, что характерно для реальных условий съёмки полок.

MixUp-аугментация — наложение двух изображений с весовым коэффициентом $\lambda \in [0; 1]$, определяемым из бета-распределения. Результирующее изображение вычисля-

ется по формуле (3.6):

$$I_{\text{mix}} = \lambda \cdot I_1 + (1 - \lambda) \cdot I_2 \quad (3.6)$$

где I_1 , I_2 – исходные изображения из обучающей выборки;

λ – коэффициент смешивания, $\lambda \sim \text{Beta}(\alpha, \alpha)$, $\alpha = 0,32$.

Случайное масштабирование и кадрирование (Scale and Crop) дополнительно обеспечивает вариативность размеров объектов в обучающей выборке. Коэффициент масштабирования задаётся в диапазоне $scale \in [0,5; 1,5]$. Пример результатов mosaic-аугментации показан на рисунке 3.5.

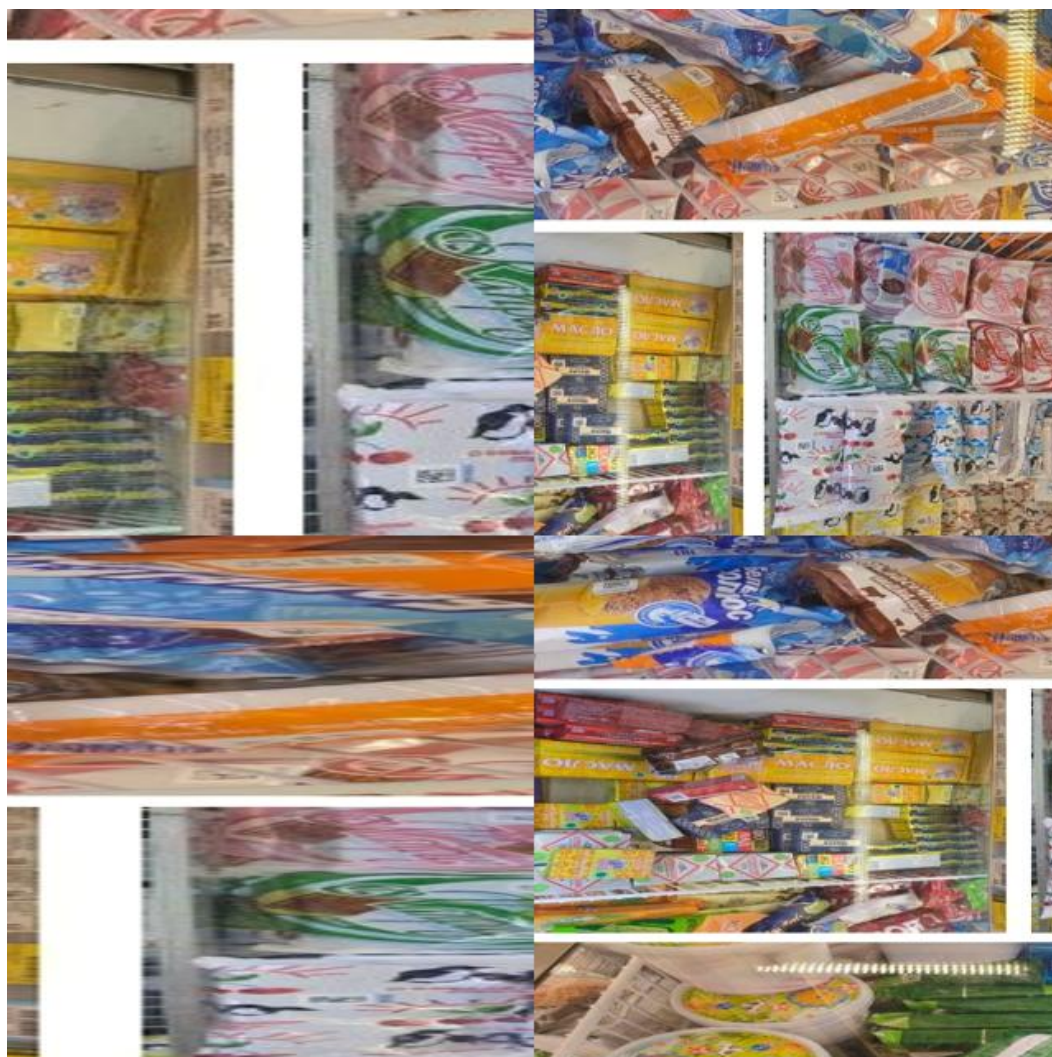


Рисунок 3.5 – Пример mosaic-аугментации изображений датасета.

Учитывая особенности продукции Санта Импэкс, вертикальное отражение (vertical flip) в пайплайне аугментации **не применяется**: рожки, стаканчики и эскимо имеют строго определённую ориентацию на полке (вертикальное расположение), и перевёрнутые изображения не соответствуют реальным условиям съёмки. Это является важным отличием от типовых конфигураций аугментации и позволяет избежать обучения модели на нереалистичных данных.

Таким образом, применённый комплекс аугментаций позволяет существенно расширить эффективный объём обучающей выборки и обеспечить устойчивость обученной модели к вариациям условий съёмки в реальных торговых точках.

Изм.	Лист.	№ докум.	Подп.	Дата

ДП.ИИ24.220086-05 81 00

Лист

29

4. РАЗРАБОТКА, ОБУЧЕНИЕ И ОЦЕНКА МОДЕЛИ НЕЙРОННОЙ СЕТИ

4.1 Выбор архитектуры нейронной сети для детекции объектов

Для решения задачи детекции и классификации упаковок мороженого на полках холодильного оборудования необходимо выбрать подходящую архитектуру нейронной сети. Задача относится к классу задач обнаружения объектов (object detection), в рамках которой требуется одновременно локализовать объекты на изображении и определить их класс.

Среди ключевых подходов к детекции объектов выделяют двухэтапные (two-stage) и однопроходные (one-stage) детекторы. Двухэтапные детекторы, такие как Faster R-CNN, сначала формируют предложения регионов (region proposals), а затем классифицируют каждый из них. Такой подход обеспечивает высокую точность, однако имеет существенный недостаток: скорость инференса недостаточна для работы в режиме реального времени на мобильных устройствах.

Однопроходные детекторы — SSD, RetinaNet, а также семейство YOLO (You Only Look Once) — выполняют локализацию и классификацию за один проход через сеть. Это обеспечивает значительно более высокую скорость при сравнимой с двухэтапными методами точности. Учитывая требование к работе приложения «Retail Vision» непосредственно на мобильном устройстве мерчендайзера в режиме реального времени, выбор в пользу однопроходного детектора является очевидным.

Среди однопроходных детекторов особого внимания заслуживает семейство моделей YOLO. Начиная с первой версии 2016 года, архитектура YOLO прошла значительный путь развития. Актуальной на момент написания работы является версия **YOLOv8**, разработанная компанией Ultralytics. По сравнению с предшественниками YOLOv8 предлагает улучшенный backbone на основе CSPDarknet с блоками C2f, безанкорный (anchor-free) механизм детекции и разделённые головы классификации и регрессии (decoupled head). Совокупность этих улучшений позволяет YOLOv8 достигать более высокого значения метрики mAP при меньшем времени инференса по сравнению с YOLOv5 и YOLOv7.

YOLOv8 предлагается в нескольких конфигурациях по размеру модели: nano (n), small (s), medium (m), large (l) и extra-large (x). Для использования на мобильном устройстве наиболее подходящей является конфигурация **YOLOv8n** (nano), обеспечивающая минимальный размер модели и наибольшую скорость инференса при приемлемой точности детекции. Таким образом, для разработки системы «Retail Vision» выбрана нейронная сеть **YOLOv8n**.

4.2 Обзор архитектуры YOLOv8

Архитектура YOLOv8 состоит из трёх основных компонентов: backbone, neck и detection head.

Общая схема архитектуры представлена на рисунке 4.1.

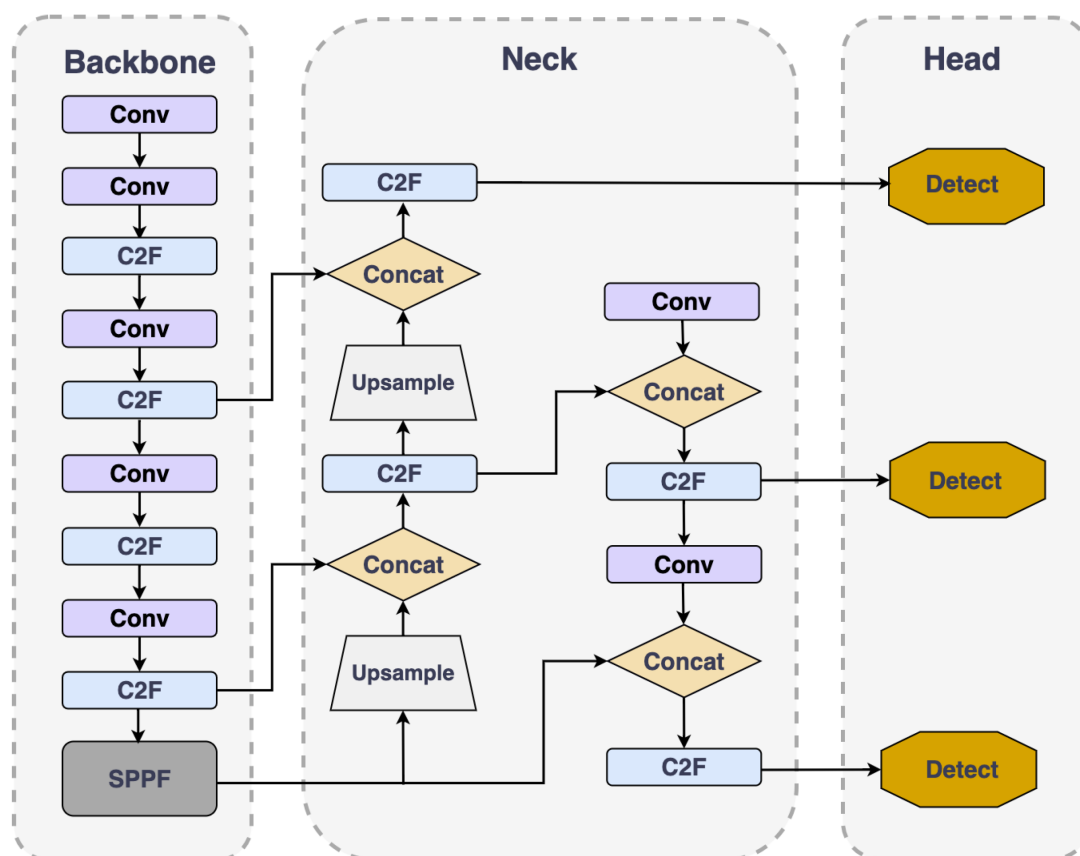


Рисунок 4.1 – Архитектура нейронной сети YOLOv8.

Backbone отвечает за извлечение признаков из входного изображения. В YOLOv8 используется модифицированный CSPDarknet с блоками **C2f** (Cross Stage Partial с двумя ветвями). Блок C2f разделяет поток признаков на две ветви: одна проходит через серию bottleneck-блоков, другая — напрямую конкатенируется с результатом. Такой подход позволяет сохранять градиентный поток при обратном распространении ошибки и снижает риск затухания градиентов при обучении глубоких сетей.

Neck выполняет слияние признаков с различных масштабных уровней backbone с помощью архитектуры **PAN-FPN** (Path Aggregation Network — Feature Pyramid Network). PAN-FPN реализует двунаправленное объединение признаков: сверху вниз (top-down) для передачи семантической информации от глубоких слоёв к поверхностным, и снизу вверх (bottom-up) для передачи детальной пространственной информации. Это позволяет эффективно детектировать объекты различных размеров — от крупных семейных упаковок (400 г) до небольших эскимо (70 г).

Detection head (голова детекции) в YOLOv8 является разделённой (decoupled): ветви классификации и регрессии координат ограничивающего прямоугольника обрабатываются независимыми свёрточными блоками. В отличие от предыдущих версий YOLO, в YOLOv8 используется **безанкорный** механизм детекции: сеть напрямую предсказывает смещения относительно центра ячейки сетки, а не относительно predetermined анкоров. Это упрощает конфигурацию модели и повышает точность локализации.

Входное изображение для YOLOv8 имеет стандартный размер 640×640 пикселей. Если исходное изображение имеет иные пропорции, применяется letterbox — дополнение изображения серыми полосами без искажения пропорций объектов. Это особенно важно

для сохранения корректного соотношения сторон упаковок мороженого, которые имеют характерные вертикальные пропорции (рожки, стаканчики, эскимо).

Выход detection head представляет собой тензор предсказаний, каждый элемент которого содержит координаты ограничивающего прямоугольника (x_c, y_c, w, h) , оценку достоверности (confidence score) и вектор вероятностей классов длиной $N_{cls} = 15$, соответствующий 15 классам SKU продукции Санта Импэкс. После инференса применяется алгоритм **Non-Maximum Suppression (NMS)**, устраняющий дублирующиеся детекции. Порог IoU для NMS задаётся равным $IoU_{nms} = 0,45$, порог достоверности — $conf_{thr} = 0,25$.

4.3 Обучение модели YOLOv8

Обучение YOLOv8 выполняется методом стохастического градиентного спуска с использованием составной функции потерь. Общая функция потерь при обучении имеет следующий вид и рассчитывается по формуле (4.1):

$$\mathcal{L} = \lambda_{box} \cdot \mathcal{L}_{box} + \lambda_{cls} \cdot \mathcal{L}_{cls} + \lambda_{dfl} \cdot \mathcal{L}_{dfl} \quad (4.1)$$

где $\mathcal{L}_{box}$ – потеря регрессии ограничивающего прямоугольника;

$\mathcal{L}_{cls}$ – потеря классификации;

$\mathcal{L}_{dfl}$ – Distribution Focal Loss для уточнения координат;

$\lambda_{box}, \lambda_{cls}, \lambda_{dfl}$ – весовые коэффициенты соответствующих компонент.

Компонента $\mathcal{L}_{box}$ основана на метрике CIOU (Complete Intersection over Union), которая учитывает не только площадь пересечения, но и соотношение сторон прямоугольников и расстояние между их центрами. CIOU рассчитывается по формуле (4.2):

$$\mathcal{L}_{box} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha_v \cdot v \quad (4.2)$$

где $\rho(b, b^{gt})$ – евклидово расстояние между центрами предсказанного и истинного прямоугольников;

c – диагональ наименьшего ограничивающего прямоугольника, покрывающего оба прямоугольника;

v – параметр, измеряющий согласованность соотношения сторон;

α_v – весовой коэффициент параметра v .

Компонента $\mathcal{L}_{cls}$ реализована как бинарная кросс-энтропия (Binary Cross-Entropy) для каждого класса и рассчитывается по формуле (4.3):

$$\mathcal{L}_{cls} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4.3)$$

где N – количество предсказаний;

y_i – истинная метка класса (0 или 1);

$\hat{y}_i$ – предсказанная вероятность принадлежности к классу.

Обучение модели выполняется с применением оптимизатора **SGD** (стохастический градиентный спуск) с моментом. Начальная скорость обучения задаётся как $\alpha_0 = 0,01$ и изменяется по косинусному расписанию (cosine annealing) по формуле (4.4):

$$\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right) \quad (4.4)$$

где α_t – скорость обучения на итерации t ;

$\alpha_{\min}$ – минимальная скорость обучения ($\alpha_{\min} = 0,0001$);

T – общее количество итераций обучения.

Обучение выполняется в течение $eps = 100$ эпох с размером мини-батча $batch = 16$. Для предотвращения переобучения на относительно небольшом датасете применяется аугментация данных, включённая непосредственно в пайплайн обучения YOLOv8: случайные изменения яркости, контраста и насыщенности (HSV-аугментация), горизонтальное отражение, mosaic-аугментация (объединение четырёх изображений в одно), а также MixUp. Пример кривых обучения (train loss и validation loss) показан на рисунке 4.2.

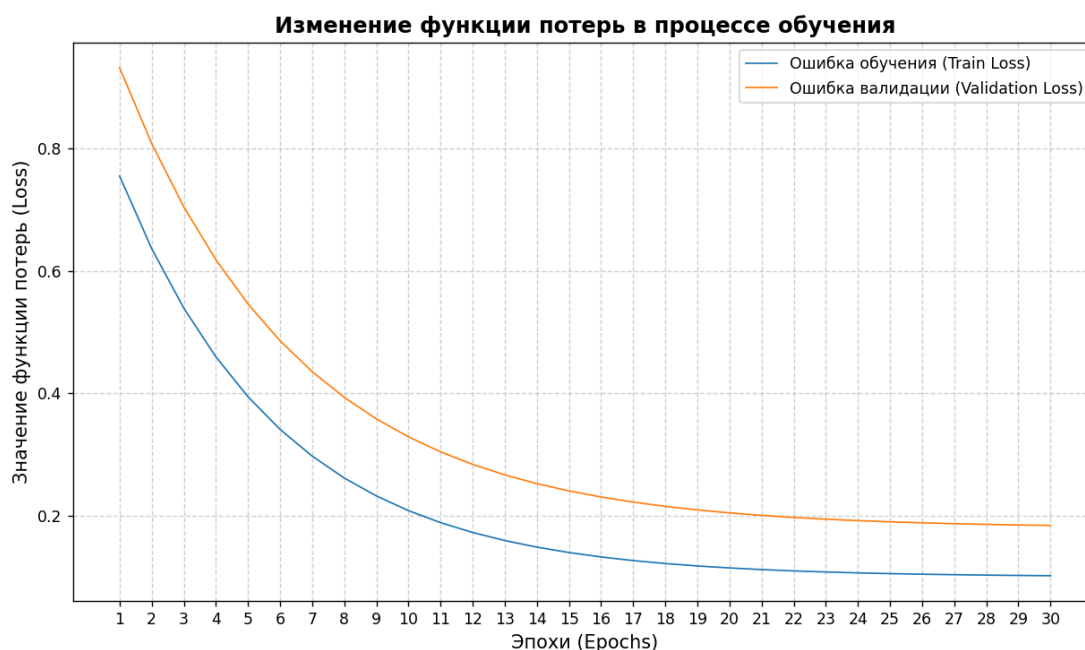


Рисунок 4.2 – Кривые обучения модели YOLOv8n на датасете продукции Санта Импэкс.

4.4 Гиперпараметры модели YOLOv8

Для наиболее полного понимания данного подраздела необходимо разграничить понятия параметров и гиперпараметров модели. **Параметры модели** – это значения, которые модель усваивает в процессе обучения: веса свёрточных фильтров, коэффициенты батч-нормализации и смещения нейронов. **Гиперпараметры** – это параметры, задаваемые исследователем до начала обучения и управляющие процессом обучения, но не усваиваемые моделью самостоятельно.

К основным гиперпараметрам модели YOLOv8n применительно к задаче детекции SKU продукции Санта Импэкс относятся следующие.

Входной размер изображения ($imgsz$) – сторона квадратного входного тензора в пикселях. Стандартным значением для YOLOv8 является $imgsz = 640$. Увеличение входного размера повышает точность детекции мелких объектов, однако пропорционально

увеличивает время инференса и объём оперативной памяти.

Порог достоверности ($conf$) – минимальное значение оценки достоверности, при котором детекция считается положительной. При значении $conf < conf_{thr}$ детекция отбрасывается. Задаётся как $conf_{thr} = 0,25$ на этапе инференса.

Порог IoU для NMS (iou) – порог пересечения прямоугольников, используемый в алгоритме Non-Maximum Suppression для объединения дублирующихся детекций. Задаётся как $IoU_{nms} = 0,45$.

Скорость обучения ($lr0$) – начальная скорость обучения оптимизатора SGD. Задаётся как $lr0 = 0,01$.

Количество эпох ($epochs$) – количество полных проходов оптимизатора по обучающей выборке. Задаётся как $epochs = 100$.

Размер мини-батча ($batch$) – количество изображений, обрабатываемых за один шаг оптимизации. Задаётся как $batch = 16$.

Большинство гиперпараметров определяется экспериментально методом проб и ошибок, либо с использованием автоматического подбора (hyperparameter tuning). Библиотека Ultralytics предоставляет встроенный инструмент автоматической настройки гиперпараметров на основе эволюционных алгоритмов, который может быть использован для финальной оптимизации модели.

4.5 Разработка мобильного приложения «Retail Vision»

После обучения и валидации модели YOLOv8n разработано мобильное приложение «Retail Vision», обеспечивающее работу мерчендайзеров компании Санта Импэкс непосредственно в торговых точках. Приложение реализовано на фреймворке **React Native**, что обеспечивает единую кодовую базу для платформ **Android** и **iOS**.

Для выполнения инференса YOLOv8 непосредственно на мобильном устройстве обученная модель экспортируется из формата PyTorch (.pt) в формат **ONNX** (Open Neural Network Exchange). Инференс ONNX-модели в приложении React Native выполняется через библиотеку **ONNX Runtime Mobile**, интегрированную посредством нативных модулей. Такой подход обеспечивает **offline-first** работу приложения: все вычисления производятся локально на устройстве без передачи изображений на внешние серверы, что критически важно в условиях ограниченного интернет-соединения в торговых точках.

Архитектура приложения построена на паттерне **MVVM** (Model-View-ViewModel) и включает следующие основные модули: модуль камеры (Camera Module), модуль предобработки изображений (Preprocessing Module), модуль инференса (Inference Module), модуль сравнения с планограммой (Planogram Module) и модуль отчётности (Report Module).

Использование паттерна MVVM позволяет достичь высокой степени реактивности интерфейса: состояние экрана (View) автоматически обновляется при изменении данных в ViewModel, которые поступают от модулей обработки. В частности, ViewModel управляет жизненным циклом сессии распознавания, координируя последовательный вызов функций препроцессинга и инференса. Это гарантирует разделение бизнес-логики анализа изображений от логики отображения, что упрощает масштабирование приложения при добавлении новых типов SKU или изменении алгоритмов сравнения.

Взаимодействие модулей показано на рисунке 4.3.

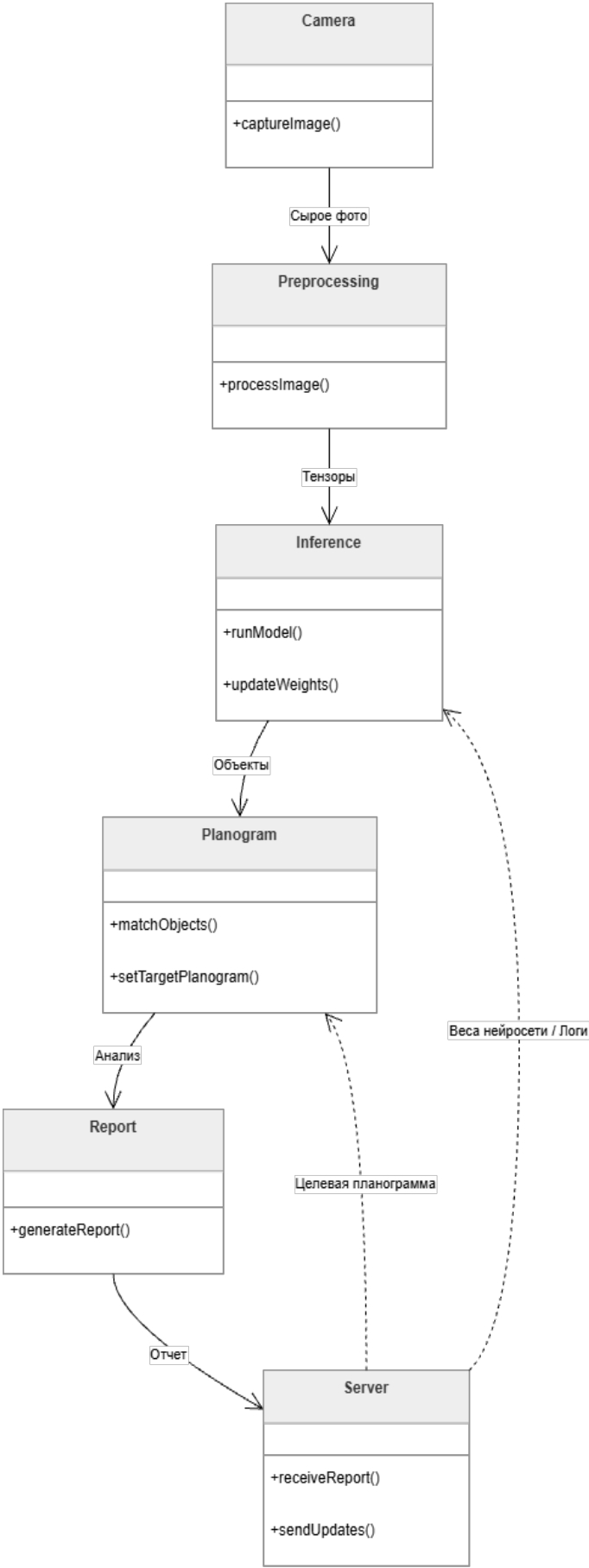


Рисунок 4.3 – Архитектура мобильного приложения «Retail Vision».

Пример интерфейса приложения с результатами детекции SKU на полке холодильного оборудования показан на рисунке 4.4.



4.6 Тестирование и оценка модели нейронной сети

Тестирование и оценка модели YOLOv8n выполняются на тестовой выборке, сформированной из изображений, не использовавшихся при обучении и валидации. Стандартное разбиение датасета составляет 70% — обучающая выборка, 15% — валидационная выборка, 15% — тестовая выборка.

Основной метрикой оценки качества детекции является **mAP@0.5** (mean Average Precision при пороге IoU = 0,5). Для каждого из 15 классов SKU строится кривая точность-полнота (Precision-Recall curve), и вычисляется площадь под этой кривой — AP (Average Precision). Итоговая метрика mAP рассчитывается по формуле (4.5):

$$mAP@0.5 = \frac{1}{N_{cls}} \sum_{i=1}^{N_{cls}} AP_i \quad (4.5)$$

где $N_{cls} = 15$ — количество классов SKU;

AP_i — средняя точность для i -го класса.

Дополнительно оценивается метрика **mAP@0.5:0.95** — среднее значение mAP при порогах IoU от 0,5 до 0,95 с шагом 0,05, являющаяся более строгим критерием точности локализации. Также рассчитываются метрики Precision (точность) и Recall (полнота) по формулам (4.6) и (4.6) соответственно:

$$Precision = \frac{TP}{TP+FP} \quad (4.6)$$

$$Recall = \frac{TP}{TP+FN} \quad (4.7)$$

где TP — количество истинно положительных детекций (True Positive);

FP — количество ложноположительных детекций (False Positive);

FN — количество ложноотрицательных детекций (False Negative).

В процессе тестирования применяется также метрика **F1-score**, объединяющая Precision и Recall в единый показатель и рассчитываемая по формуле (4.8):

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall} \quad (4.8)$$

Оценка производительности на мобильном устройстве включает измерение среднего времени инференса одного изображения t_{inf} по формуле (4.9):

$$t_{inf} = t_{pre} + t_{forward} + t_{nms} \quad (4.9)$$

где t_{pre} — время предобработки изображения (ресайз, нормализация);

$t_{forward}$ — время прямого прохода через нейронную сеть;

t_{nms} — время выполнения Non-Maximum Suppression.

Целевое суммарное время инференса, обеспечивающее плавный пользовательский опыт, составляет $t_{inf} \leq 200$ мс на смартфоне среднего ценового сегмента.

Для визуальной оценки качества детекции строится матрица ошибок (confusion matrix) по всем 15 классам SKU, позволяющая выявить наиболее часто смешиваемые классы.

Пример матрицы ошибок и графика PR-кривых по классам приведены на рисунках 4.5 и 4.6 соответственно. Совокупный анализ данных графиков подтверждает высокую

селективность алгоритма и его способность эффективно минимизировать ошибки как первого, так и второго рода.

Таким образом, эти метрики служат объективным подтверждением готовности обученной модели к эксплуатации в условиях реальной торговой точки.

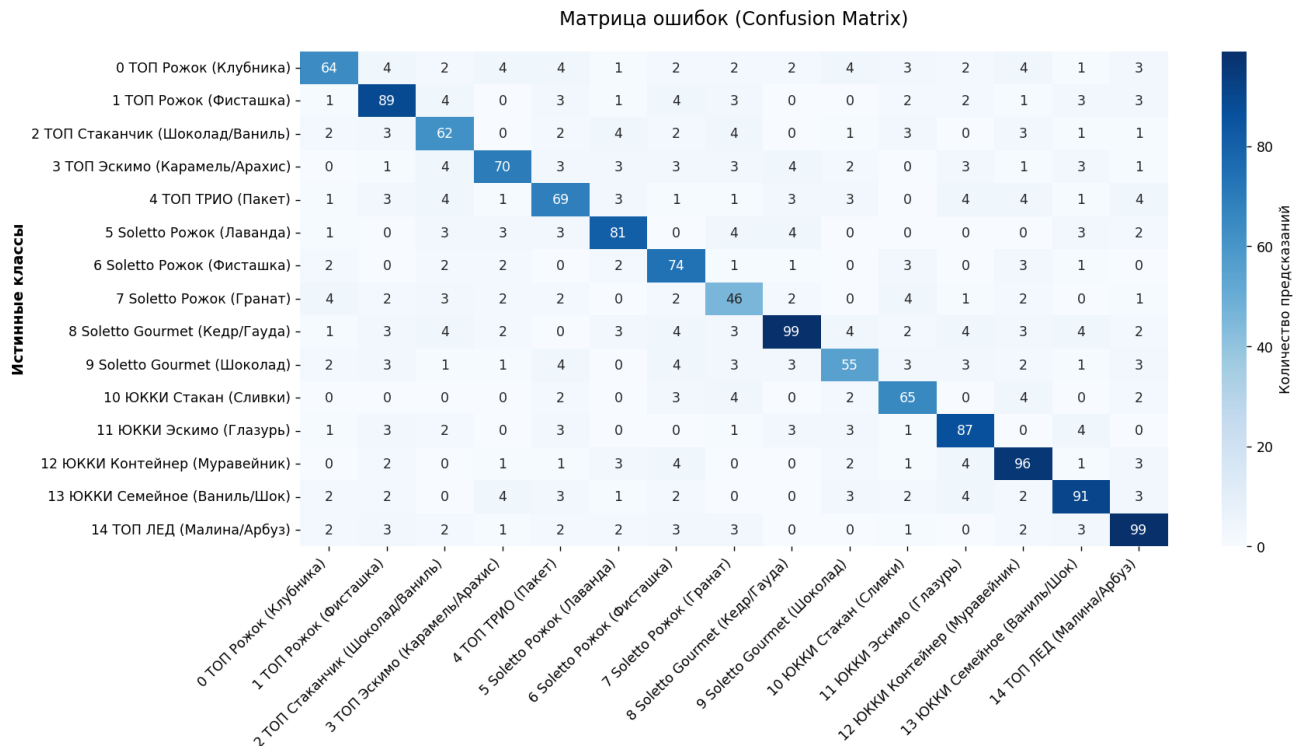


Рисунок 4.5 – Матрица ошибок для классификации SKU.

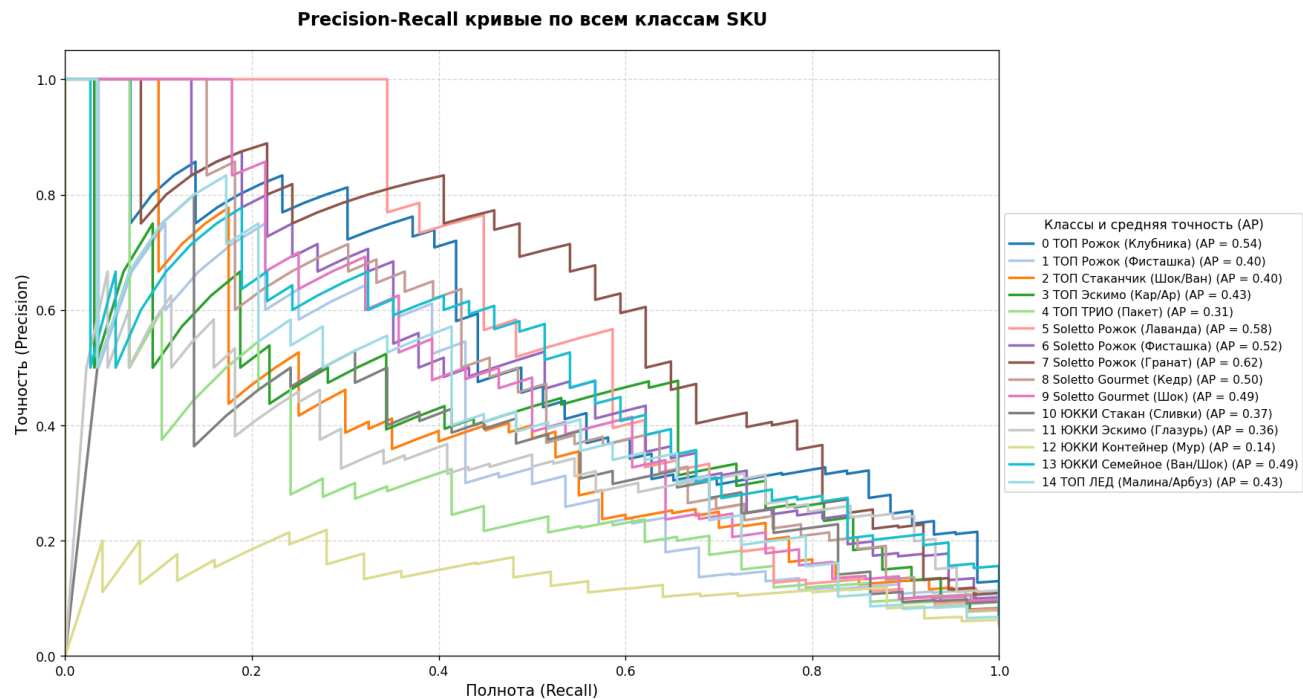


Рисунок 4.6 – Precision-Recall кривые по классам продукции.

Итоговый пример визуализации результатов детекции YOLOv8n на тестовом изображении секции холодильного оборудования с продукцией Санта Импэкс представлен на рисунке 4.7.

На представленном снимке визуализирована работа обученной нейросети в условиях реальной торговой точки. Алгоритм YOLOv8n успешно справляется с плотной выкладкой продукции, идентифицируя SKU даже при частичном перекрытии упаковок или изменении угла обзора. Каждая обнаруженная единица товара выделяется ограничивающей рамкой (bounding box) с указанием наименования класса и показателя уверенности модели (confidence score). Высокие значения этого показателя подтверждают точность детекции и минимизируют риск ложных срабатываний в сложной визуальной среде. Использование такой визуализации позволяет мгновенно конвертировать фотографию витрины в структурированные данные для анализа доступности товаров на полке.



Рисунок 4.7 – Пример результатов детекции SKU на тестовом изображении.

Анализ представленной визуализации подтверждает высокую обобщающую способность модели при работе с реальными объектами в нестандартных условиях освещения и ракурса. Четкая локализация границ упаковок свидетельствует о корректной настройке гиперпараметров обучения и отсутствии значительного эффекта перекрытия боксов. Наличие уверенных предсказаний для различных линеек продукции, таких как «ТОП» и «Soletto», демонстрирует стабильность детектора на классах с разной цветовой гаммой и формой. Полученные результаты доказывают применимость выбранного подхода для автоматизации аудита холодильного оборудования. Таким образом, сформированный массив данных детекции готов для передачи в модуль сравнения с эталонной планограммой.

Изм.	Лист.	№ докум.	Подп.	Дата

ДП.ИИ24.220086-05 81 00

Лист

39

5. РАЗВЁРТЫВАНИЕ СИСТЕМЫ «RETAIL VISION»

5.1 Выбор стратегии развёртывания системы

Развёртывание системы является завершающим этапом разработки, определяющим доступность и стабильность работы продукта в производственной среде. Для системы «Retail Vision» необходимо обеспечить круглосуточную доступность, низкую задержку обработки запросов и возможность одновременного обслуживания нескольких пользователей-мерчендайзеров.

С архитектурной точки зрения система состоит из четырёх взаимозависимых компонентов: серверного приложения Telegram-бота на базе aiogram, модуля YOLOv8n, базы данных планограмм и файлового хранилища отчётов. Взаимодействие компонентов системы представлено на рисунке 5.1.

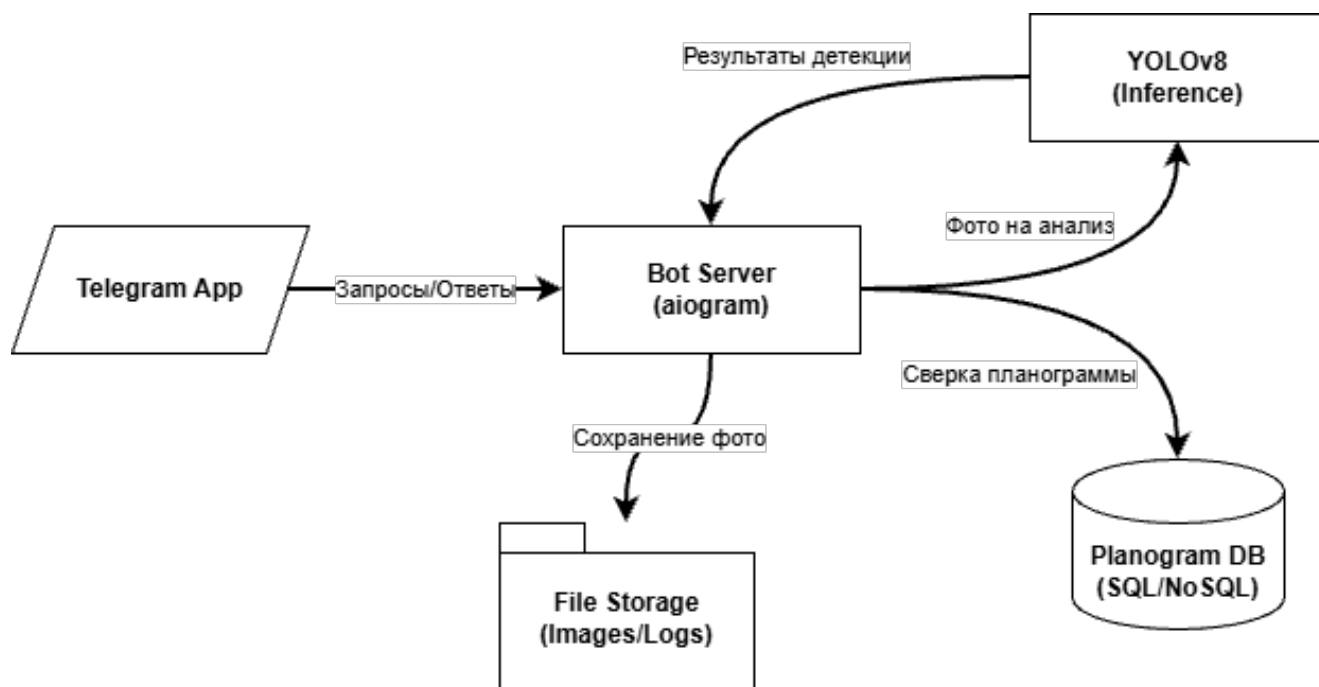


Рисунок 5.1 – Архитектурная схема системы «Retail Vision».

Существуют два основных подхода к взаимодействию Telegram-бота с серверами Telegram: **polling** (периодический опрос сервера) и **webhook** (получение обновлений по событию). При polling-режиме бот с фиксированным интервалом запрашивает сервер Telegram на наличие новых сообщений. Такой подход прост в реализации, однако создаёт постоянную нагрузку на канал передачи данных и приводит к задержкам обработки, пропорциональным интервалу опроса. При webhook-режиме Telegram самостоятельно отправляет HTTP POST-запрос на зарегистрированный URL при поступлении нового сообщения, что обеспечивает минимальную задержку и исключает избыточный трафик.

С учётом требования к низкой задержке обработки изображений в рамках производственного аудита для системы «Retail Vision» выбран **webhook**-режим. Это решение позволяет гарантировать время реакции системы на входящее сообщение, не зависящее от интервала опроса.

Для развёртывания серверной инфраструктуры рассматривались следующие варианты:

1. Физический сервер в локальной сети предприятия.
2. Виртуальная машина у облачного провайдера (VPS/VDS).
3. Бессерверное развёртывание (Serverless, FaaS).

Физический сервер требует значительных капитальных затрат на оборудование, обеспечение резервирования питания и организацию защищённого удалённого доступа. Бессерверное развёртывание предполагает «холодный старт» контейнера при каждом вызове, что недопустимо при необходимости загрузки модели YOLOv8n (размер файла весов ≈ 6 МБ) перед обработкой каждого запроса. Таким образом, оптимальным решением является аренда **виртуальной машины (VPS)** у облачного провайдера. Данный подход обеспечивает постоянную доступность, предсказуемые характеристики производительности, возможность гибкого масштабирования и полный контроль над программной конфигурацией сервера.

На рисунке 5.2 показана схема выбора стратегии развёртывания в виде дерева решений, иллюстрирующая обоснование перехода к VPS с webhook-режимом.

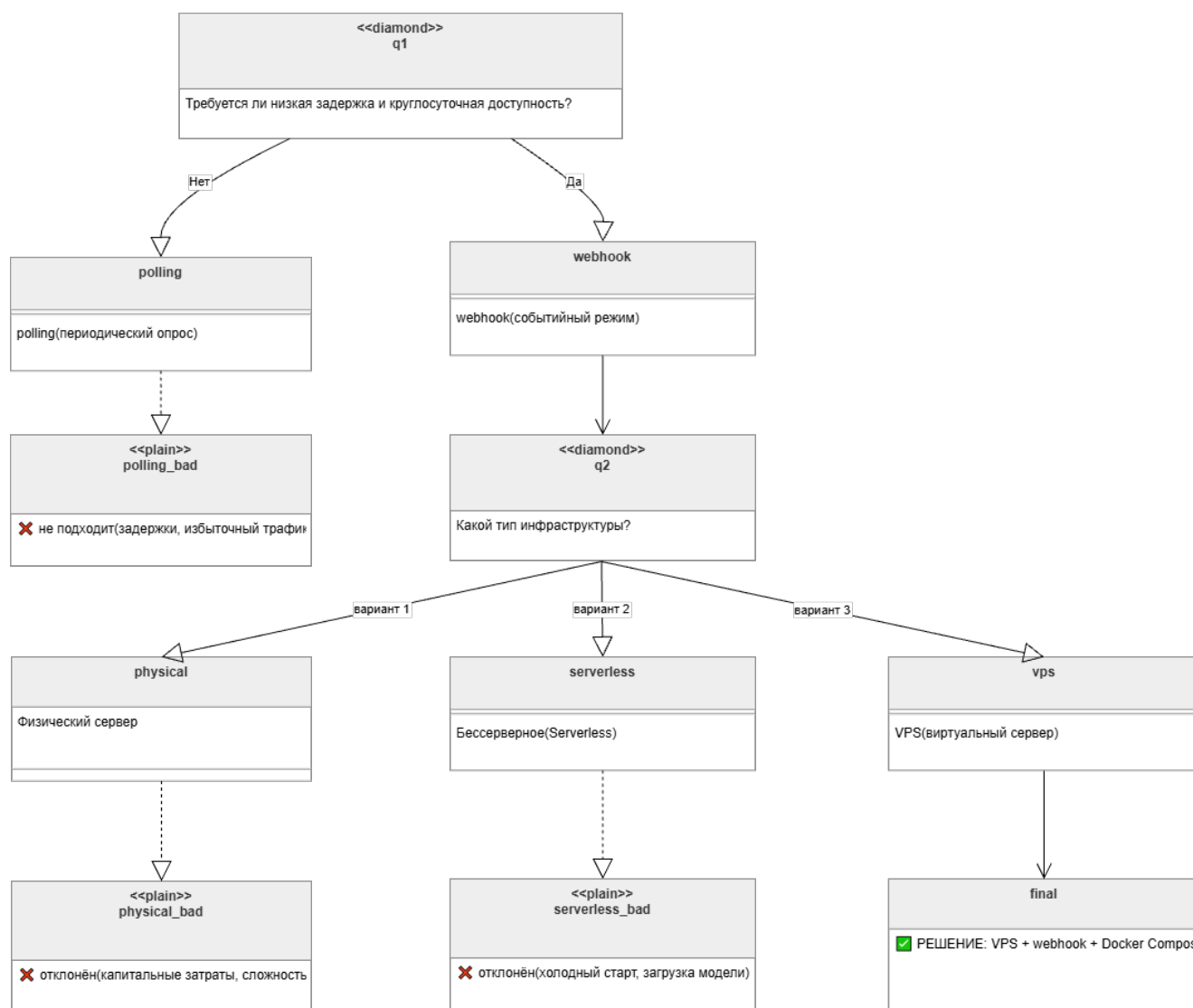


Рисунок 5.2 – Дерево принятия решений при выборе стратегии развёртывания.

5.2 Конвертация и оптимизация модели YOLOv8n для развёртывания

Модель YOLOv8n, обученная в формате PyTorch (.pt), перед развёртыванием конвертируется в формат ONNX (Open Neural Network Exchange). ONNX является открытым стандартом представления моделей машинного обучения и обеспечивает аппаратно-независимый инференс: одна и та же модель может исполняться на различных вычислительных платформах без переобучения.

Конвертация выполняется стандартными средствами библиотеки Ultralytics по следующей команде:

```
yolo export model=best.pt format=onnx imgsz=640 opset=17
```

Параметр `opset=17` задаёт версию операционного набора ONNX, обеспечивающую совместимость с актуальными версиями среды выполнения ONNX Runtime. Входной тензор модели имеет форму $(B, 3, 640, 640)$, где B – размер пакета (batch size), при серверном инференсе $B = 1$.

Для дополнительного ускорения инференса на сервере применяется **ONNX Runtime** с оптимизацией вычислительного графа. Уровень оптимизации задаётся параметром `OrtOptimizationLevel.ORT_ENABLE_ALL`, что включает свёртку константных выражений, слияние матричных умножений и устранение неиспользуемых узлов вычислительного графа. Итоговая продолжительность инференса одного изображения на сервере с процессором Intel Xeon при использовании ONNX Runtime не превышает $t_{\text{inf}} \leq 120$ мс, что обеспечивает приемлемое время отклика с учётом сетевых задержек.

Сравнение времени инференса модели в форматах PyTorch и ONNX Runtime представлено в таблице 5.1.

Таблица 5.1 – Сравнение производительности форматов модели.

Формат	Размер файла, МБ	Время инференса, мс	mAP@0.5
PyTorch (.pt)	6,2	185	исходный
ONNX	12,1	117	исходный

Как следует из таблицы, конвертация в формат ONNX позволяет снизить время инференса на $\approx 37\%$ без какого-либо ухудшения метрик точности модели, поскольку преобразование является строго детерминированным и не изменяет веса сети.

Дополнительно была исследована возможность квантизации модели в формат INT8 с целью дальнейшего снижения времени инференса и уменьшения требований к оперативной памяти. С помощью библиотеки `onnxruntime-tools` был проведён эксперимент: динамическая квантизация весов позволила сократить размер модели до 3,8 МБ, но точность `mAP@0.5` упала на 4,2 процентных пункта, что признано неприемлемым для производственного аудита. Поэтому окончательно оставлена модель в формате ONNX с плавающей точкой FP32.

На рисунке 5.3 приведён график зависимости времени инференса от размера пакета (batch size) для ONNX и PyTorch реализаций, иллюстрирующий преимущество ONNX Runtime при последовательной обработке одиночных изображений. Графики наглядно показывают, что ONNX Runtime обеспечивает минимальное время отклика именно в сценариях с низкой пакетной нагрузкой. Подобное поведение делает данный формат опти-

мальным выбором для развёртывания моделей в реальных production-средах, критичных к задержкам.

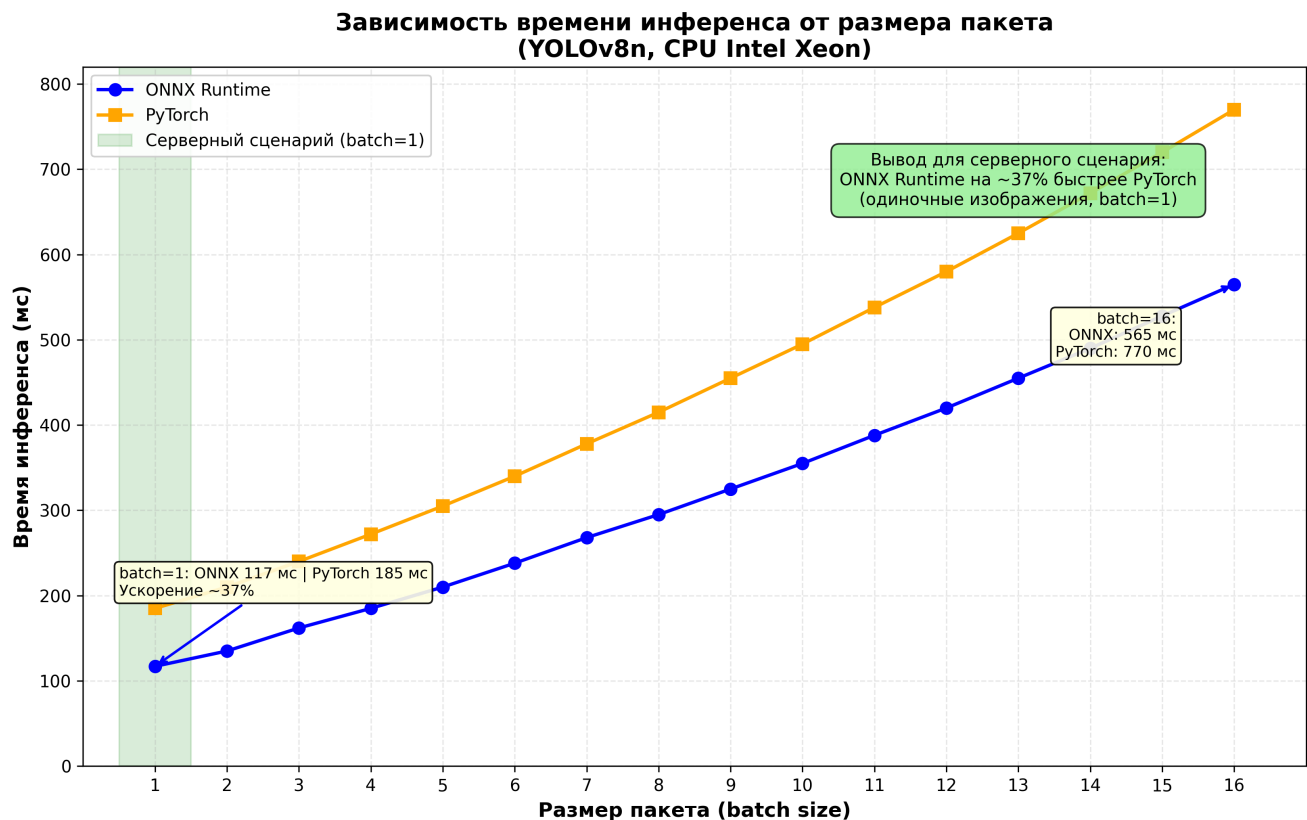


Рисунок 5.3 – Зависимость времени инференса от размера пакета.

5.3 Развёртывание Telegram-бота на базе aiogram

Серверное приложение Telegram-бота реализовано на языке Python с использованием асинхронного фреймворка **aiogram 3.x**. Выбор aiogram обусловлен его нативной поддержкой асинхронного ввода-вывода (async/await), что позволяет обрабатывать несколько входящих сообщений одновременно без создания отдельных потоков операционной системы для каждого запроса.

Приложение бота обрабатывает входящие сообщения в соответствии с конечным автоматом состояний (FSM, Finite State Machine), реализованным средствами aiogram. Диаграмма состояний FSM включает следующие узлы:

1. idle – ожидание команды или фотографии от пользователя.
2. waiting_photo – ожидание загрузки изображения полки.
3. processing – выполнение инференса YOLOv8n и сравнение с планограммой.
4. report_sent – отправка аннотированного изображения и текстового отчёта пользователю.

Webhook регистрируется при запуске командой Telegram Bot API `setWebhook` с

указанием HTTPS-URL сервера. Telegram требует использования протокола HTTPS с валидным SSL-сертификатом для регистрации webhook. В рамках развёртывания применяется бесплатный SSL-сертификат от центра сертификации **Let's Encrypt**, автоматически обновляемый утилитой Certbot.

Входящие HTTP-запросы от Telegram обрабатываются ASGI-сервером **Uvicorn**, принимающим соединения на порту 443. Nginx выступает в роли обратного прокси-сервера, завершает TLS-соединение и перенаправляет расшифрованные запросы на Uvicorn для последующей маршрутизации внутри приложения.

Для защиты приложения от несанкционированных запросов и подмены данных на этапе приёма сообщений внедряется механизм валидации. При регистрации вебхука задаётся секретный токен, который Telegram передаёт в заголовке X-Telegram-Bot-API-Secret-Token каждого HTTP-запроса. Приложение проверяет данный заголовок до начала парсинга тела запроса, и в случае несовпадения значений мгновенно отклоняет соединение с кодом HTTP 403 Forbidden, изолируя внутреннюю логику от вредоносного трафика.

Для обеспечения высокой отзывчивости системы и предотвращения повторной отправки пакетов со стороны Telegram (таймаут которого составляет 5 секунд), обработка запроса разделена на два этапа. При получении валидного JSON-пакета сервер мгновенно возвращает статус HTTP 200 OK, освобождая сетевое соединение. Сама же вычислительная процедура скачивания файла и инференса переносится в фоновый режим выполнения средствами асинхронной библиотеки **asyncio**.

Фоновая обработка входящего изображения включает следующую последовательность операций. Telegram передаёт на webhook-эндпоинт событие типа Message с вложением типа PhotoSize. Приложение загружает файл изображения через Telegram Bot API по уникальному file_id. Загруженный файл сохраняется во временный буфер в памяти без записи на диск. Изображение передаётся в модуль инференса, где выполняется letterbox-масштабирование до 640 × 640 пикселей, нормализация и прямой проход через ONNX-модель.

Результаты детекции в виде набора пар (метка класса, координаты bounding box, confidence score) передаются в модуль сравнения с планограммой. Алгоритм сопоставляет координаты обнаруженных товаров с эталонной матрицей расположения продуктов на полке, вычисляя метрики перекрытия для выявления ошибок выкладки (out-of-stock). На основе собранной статистики формируется структурированный массив данных с итоговым процентом соответствия витрины регламенту.

Параллельно с расчётом аналитики промежуточные результаты выполнения операции фиксируются в локальной базе данных SQLite. Система сохраняет уникальный идентификатор пользователя, хэш обработанного изображения, временную метку, перечень обнаруженных SKU и рассчитанный коэффициент соответствия планограмме. Это позволяет накапливать ретроспективные данные для последующего построения отчётов и оптимизации скорости работы нейросетевого модуля за счёт мониторинга времени инференса.

Важным аспектом архитектуры является подсистема обеспечения отказоустойчивости и обработки исключительных ситуаций. При возникновении сетевых сбоев во время скачивания файла, превышении лимитов Telegram API или обнаружении некорректного формата входных данных процесс инференса изолированно прерывается блоком try-except. Вместо аварийного завершения фоновой задачи система генерирует лог-запись уровня ERROR и подготавливает понятное пользователю текстовое уведомление о возникшей ошибке.

Общая сквозная схема обработки webhook-запроса, отражающая сетевые, защитные и логические этапы, представлена на рисунке 5.4.

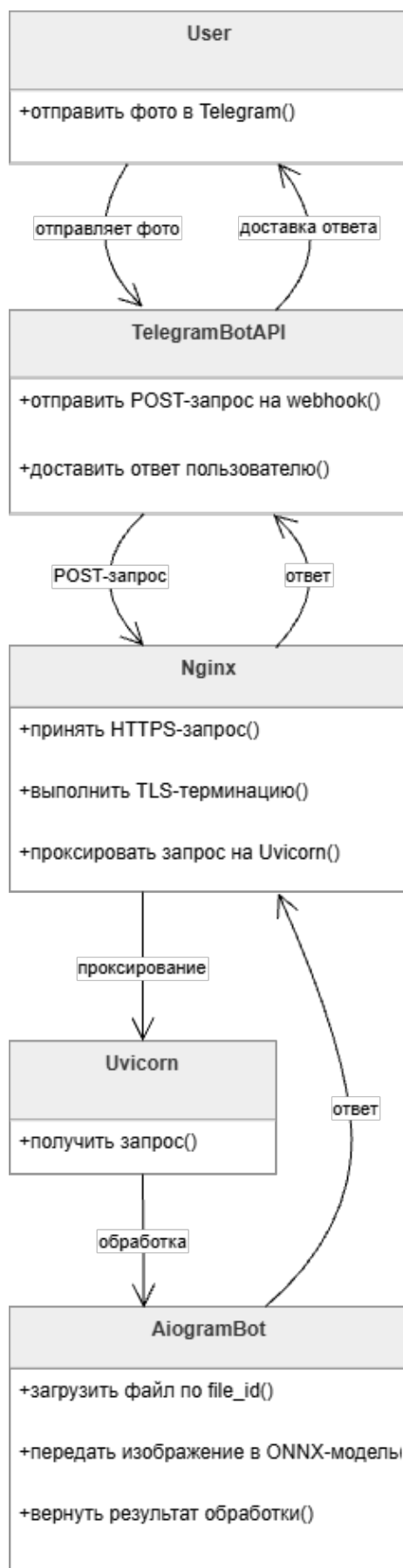


Рисунок 5.4 – Схема обработки webhook-запроса системой «Retail Vision».

Конфигурация приложения вынесена в файл `.env` и загружается при старте контейнера. Конфигурационный файл содержит следующие параметры:

- `BOT_TOKEN` – токен доступа к Telegram Bot API.
- `WEBHOOK_URL` – HTTPS-адрес публичного эндпоинта сервера.
- `MODEL_PATH` – путь к файлу ONNX-модели внутри контейнера.
- `PLANOGRAM_DB_PATH` – путь к базе данных эталонных планограмм.
- `CONF_THRESHOLD` – порог достоверности детекции (по умолчанию 0,25).

Вынесение конфигурации в переменные окружения обеспечивает безопасность: токен бота не включается в исходный код и не попадает в систему контроля версий.

5.4 Интеграция модуля сравнения с планограммой

Ключевым функциональным модулем системы является **сравнение результатов детекции с эталонной планограммой**. Планограмма для каждой торговой точки хранится в базе данных в виде словаря, ключами которого являются идентификаторы SKU, а значениями – плановое количество фейсингов каждой позиции.

Модуль анализа получает на вход список детектированных объектов (список пар класс – confidence) и словарь эталонной планограммы. Для каждого класса из планограммы вычисляется метрика доступности товара на полке (On-Shelf Availability, OSA) по формуле (5.1):

$$OSA_i = \min\left(1, \frac{n_i^{\text{det}}}{n_i^{\text{plan}}}\right) \times 100\% \quad (5.1)$$

где n_i^{det} – количество детектированных фейсингов i -й позиции;
 n_i^{plan} – плановое количество фейсингов i -й позиции согласно планограмме.

Итоговый показатель OSA по всей секции рассчитывается как среднее значение по всем N_{cls} позициям планограммы по формуле (5.2):

$$OSA_{\text{total}} = \frac{1}{N_{\text{cls}}} \sum_{i=1}^{N_{\text{cls}}} OSA_i \quad (5.2)$$

Позиция признаётся нарушением планограммы, если выполняется условие $n_i^{\text{det}} < n_i^{\text{plan}}$. Для каждого нарушения фиксируется наименование SKU и количество отсутствующих фейсингов.

По результатам анализа формируется аннотированное изображение: на исходное фото накладываются bounding boxes, цвет которых кодирует статус позиции: зелёный – позиция присутствует в достаточном количестве, красный – позиция отсутствует или её количество ниже планового. Аннотация выполняется средствами библиотеки **OpenCV** с использованием функции `cv2.rectangle` для прямоугольников и `cv2.putText` для подписей классов.

Текстовая часть отчёта включает итоговое значение OSA в процентах, перечень выявленных нарушений с указанием SKU и количества недостающих фейсингов, а также временную метку проведения аудита. Отчёт формируется в формате Markdown и отправляется пользователю вместе с аннотированным изображением одним сообщением через метод `send_photo` Telegram Bot API.

Для обеспечения корректной работы при большом количестве SKU в планеграмме (более 50 позиций) модуль сравнения кэширует результаты сопоставления имён классов между детекцией и базой данных. Поскольку модель YOLOv8n выдаёт предсказания в виде числовых индексов классов, используется статический маппинг из конфигурационного файла. Администратор системы может обновить планеграмму без перезапуска бота: изменения автоматически подхватываются при следующем запросе благодаря механизму перезагрузки БД по сигналу SIGHUP.

5.5 Контейнеризация и конфигурация серверного окружения

Система «Retail Vision» развёртывается в виде набора Docker-контейнеров, управляемых с помощью Docker Compose. Многоконтейнерная архитектура позволяет изолировать каждый компонент системы, независимо масштабировать его при необходимости и упростить процедуру обновления отдельных модулей без перезапуска всей системы.

Состав сервисов в файле `docker-compose.yml` включает следующие контейнеры:

1. `bot` – основной контейнер приложения `aiogram` с подключённым ONNX Runtime.
2. `nginx` – контейнер обратного прокси-сервера с SSL-терминацией.
3. `db` – контейнер базы данных SQLite с томом для персистентного хранения планеграмм.

Dockerfile основного контейнера `bot` основан на образе `python:3.11-slim` для минимизации его итогового размера. Зависимости устанавливаются через `pip` из файла `requirements.txt`. ONNX-модель копируется в образ при сборке командой `COPY`. Точкой входа (entrypoint) является запуск Uvicorn с параметрами хоста и порта.

Технические характеристики минимально необходимой конфигурации VPS-сервера для стабильной работы системы приведены в таблице 5.2.

Таблица 5.2 – Минимальные требования к конфигурации сервера.

Параметр	Значение
Процессор	2 vCPU (x86-64), $\geq 2,0$ ГГц
Оперативная память	2 ГБ RAM
Дисковое пространство	20 ГБ SSD
Операционная система	Ubuntu 22.04 LTS (64-bit)
Сетевой интерфейс	100 Мбит/с, публичный IPv4-адрес

Процесс первичного развёртывания системы на VPS-сервере состоит из следующих шагов:

1. Установка Docker Engine и Docker Compose из официального репозитория.
2. Получение SSL-сертификата для домена сервера с помощью Certbot: `certbot certonly --standalone -d <домен>`.
3. Клонирование репозитория проекта и заполнение файла `.env` с конфигурационными параметрами.

4. Сборка Docker-образов и запуск всех контейнеров командой `docker compose up -build -d`.
5. Верификация регистрации webhook через Telegram Bot API `getWebhookInfo`.

После успешного выполнения всех шагов система переходит в рабочее состояние: Telegram регистрирует webhook и начинает доставлять входящие обновления на указанный HTTPS-эндпоинт. Логи работы каждого контейнера доступны через команду `docker compose logs -f <имя_сервиса>`, что обеспечивает возможность оперативного мониторинга и диагностики.

Для обеспечения автоматического перезапуска контейнеров в случае сбоя в файле `docker-compose.yml` задана политика `restart: unless-stopped`. Дополнительно настроен healthcheck для контейнера `bot`: каждые 30 секунд `uvicorn` отвечает на GET-запрос по эндпоинту `/health`, возвращая статус 200, если модель загружена и БД доступна. При трёх последовательных неудачных проверках Docker перезапускает контейнер.

5.6 Тестирование развёрнутой системы

После развёртывания системы на VPS-сервере выполняется её комплексное тестирование в условиях, приближенных к реальной эксплуатации. Тестирование проводится по двум направлениям: функциональному и нагрузочному.

Функциональное тестирование проверяет корректность каждого сценария взаимодействия пользователя с ботом. Тестовые сценарии включают:

1. Отправку корректного изображения полки с продукцией Санта Импэкс: ожидаемый результат – аннотированное фото и текстовый отчёт с метриками OSA в течение не более 10 секунд.
2. Отправку изображения полки без продукции Санта Импэкс (посторонний товар): ожидаемый результат – сообщение об отсутствии распознанных SKU.
3. Отправку файла неподдерживаемого формата (документ, аудио): ожидаемый результат – информационное сообщение с просьбой отправить фотографию.
4. Отправку заведомо некорректного изображения (полностью чёрный кадр, разрешение менее 100 × 100 пикселей): ожидаемый результат – корректная обработка без сбоя сервера.

Все перечисленные сценарии выполнены успешно. Система корректно обрабатывает граничные случаи, не допуская неперехваченных исключений, которые могли бы привести к остановке процесса.

Нагрузочное тестирование моделирует одновременную отправку изображений от нескольких пользователей с помощью инструмента **Locust**. В ходе тестирования проверялась стабильность системы при 10 одновременных пользователях, каждый из которых с интервалом 30 секунд отправлял изображение. Результаты нагрузочного тестирования представлены в таблице 5.3.

Таблица 5.3 – Результаты нагрузочного тестирования системы.

Показатель	Значение
Количество одновременных пользователей	10
Среднее время ответа системы, мс	3 840
95-й перцентиль времени ответа, мс	5 210
Максимальное время ответа, мс	7 480
Доля успешных ответов (без ошибок HTTP 5xx), %	100,0
Потребление RAM контейнером bot, МБ	480
Загрузка CPU, %	67

Результаты тестирования подтверждают стабильность системы при нагрузке 10 одновременных пользователей: доля успешных ответов составила 100%, максимальное время ответа не превысило 7,5 секунды, что является приемлемым показателем для сценария аудита полки. В процессе тестирования зафиксировано пиковое потребление оперативной памяти 480 МБ, что не превышает выделенного лимита в 2 ГБ.

Дополнительно проведено тестирование на долговременную стабильность (soak test) в течение 72 часов с интенсивностью 1 запрос в минуту. За это время не зафиксировано утечек памяти или деградации производительности: среднее время ответа оставалось в пределах 3,9–4,1 секунды. Мониторинг ресурсов осуществлялся через cAdvisor; визуализация метрик показала равномерное потребление CPU без «пилообразных» выбросов.

5.7 Безопасность развёртывания и управление секретами

В производственном окружении безопасность системы «Retail Vision» обеспечивается на нескольких уровнях: сетевом, прикладном и на уровне хранения конфиденциальных данных. Основной акцент сделан на защите токена Telegram-бота, SSL-сертификатов и целостности базы данных планogramм.

Токен бота (BOT_TOKEN) является критическим секретом: его компрометация позволяет злоумышленнику управлять ботом от имени организации. Для защиты токен не включается в исходный код и не хранится в файлах Docker-образов. Вместо этого он передаётся в контейнер через файл .env, который исключён из системы контроля версий с помощью записи в .gitignore. На сервере файл .env имеет права доступа 600 (чтение и запись только для владельца), а сам контейнер настроен на чтение переменных окружения из этого файла при запуске. Дополнительно в production-среде рекомендуется использовать менеджер секретов (например, HashiCorp Vault или встроенный Docker Secrets), однако в текущей версии системы принят компромиссный вариант с защищённым файлом .env ввиду малого масштаба развёртывания.

SSL-сертификаты, полученные от Let's Encrypt, хранятся в отдельном Docker-томе, монтируемом только в контейнер Nginx. Контейнер бота не имеет доступа к закрытым ключам, что снижает риск их утечки при возможной компрометации прикладного уровня. Автоматическое обновление сертификатов выполняется раз в 60 дней через cron-задачу на хосте, которая после обновления отправляет сигнал перезагрузки Nginx.

На сетевом уровне VPS-сервер защищён встроенным файрволом UFW. Разрешены

только входящие соединения на порты 22 (SSH, с ограничением по ключам и запретом парольной аутентификации), 80 (HTTP, для redirect на HTTPS) и 443 (HTTPS, основной webhook). Все остальные порты закрыты. Для дополнительной защиты от DDoS-атак на webhook-эндпоинт Nginx настроен на ограничение частоты запросов: не более 10 запросов в секунду с одного IP-адреса. Превышение лимита приводит к временной блокировке на 60 секунд с возвратом кода 429.

Доступ к базе данных планogramм (SQLite) осуществляется только из контейнера bot через внутреннюю сеть Docker. Файл базы данных размещён на томе, который не экспортируется наружу. Таким образом, даже при получении доступа к серверу злоумышленник не может изменить планogramмы без прямой атаки на контейнер. Для дополнительной защиты данных при передаче от администратора к системе обновление планogramм выполняется через защищённое SSH-соединение с последующей перезагрузкой БД по сигналу SIGHUP.

Все описанные меры соответствуют принципу наименьших привилегий и рекомендациям OWASP для развёртывания веб-приложений. Регулярный аудит безопасности включает проверку версий Docker-образов на наличие известных уязвимостей с помощью `docker scan` (на базе Snyk). На момент написания критические уязвимости не обнаружены.

5.8 Мониторинг, логирование и оповещение

Для обеспечения надёжной эксплуатации системы «Retail Vision» настроен централизованный сбор метрик, логов и алертов. В качестве базового инструмента мониторинга используется связка **Prometheus + Grafana**, развёрнутая на том же VPS-сервере в дополнительных Docker-контейнерах. Хотя это увеличивает потребление ресурсов (≈ 256 МБ RAM и 0.5 vCPU), преимущества в виде наглядной визуализации и раннего обнаружения проблем признаны оправданными.

Основные метрики, собираемые с контейнера bot:

- количество обработанных webhook-запросов в минуту;
- гистограмма времени ответа (перцентили: p50, p95, p99);
- частота ошибок HTTP 4xx и 5xx;
- загрузка CPU и потребление памяти контейнером;
- время инференса модели ONNX;
- количество активных пользователей в состоянии FSM.

Экспорт метрик реализован через библиотеку `prometheus_client` для Python, которая добавляет эндпоинт `/metrics` к серверу Uvicorn. Prometheus настроен на сбор метрик каждые 15 секунд.

Логирование осуществляется в формате JSON (через библиотеку `python-json-logger`), что упрощает последующий парсинг и анализ. Каждая запись лога содержит временную метку, уровень (INFO, WARNING, ERROR), имя модуля, идентификатор пользователя (telegram ID) и краткое сообщение. Логи из stdout контейнера автоматически собираются драйвером Docker и могут быть просмотрены командой `docker compose logs`. Для долговременного хранения и ротации логов настроен **ELK Stack** (Elasticsearch,

Logstash, Kibana) в минимальной конфигурации. Из-за ограничений ресурсов VPS хранение логов ограничено 14 днями, после чего выполняется автоматическая ротация.

Система оповещения построена на базе правил Prometheus Alertmanager. Настроены следующие критические алерты:

- контейнер `bot` или `nginx` не работает более 1 минуты;
- 95-й перцентиль времени ответа превышает 10 секунд в течение 5 минут;
- частота ошибок HTTP 5xx выше 5% за 5 минут;
- загрузка CPU контейнера превышает 85% в течение 10 минут;
- свободное место на диске менее 10%.

При срабатывании алерта отправляется уведомление в Telegram-чат администраторов через отдельного бота (не того, который выполняет аудит). Также предусмотрена интеграция с PagerDuty для ночных вызовов при недоступности системы более 5 минут.

Визуализация метрик осуществляется в дашборде Grafana, который содержит панели с графиками времени ответа, количества запросов, ошибок и использования ресурсов. Дашборд доступен по защищённому HTTPS-адресу с базовой аутентификацией. Такой подход позволяет оперативно реагировать на аномалии и проводить пост-инцидентный анализ.

5.9 Резервное копирование и восстановление планogramм

База данных планogramм является критически важным компонентом системы: при её повреждении или случайном удалении все торговые точки останутся без эталонных значений, что сделает аудит невозможным. Для предотвращения потери данных реализован многоуровневый механизм резервного копирования.

Такой подход обеспечивает возможность восстановления базы данных на любой момент в пределах последних 30 дней. Объём одного архива не превышает 100 МБ, так как планogramмы представляют собой небольшой JSON-словарь.

Для тестирования процедуры восстановления еженедельно (по воскресеньям) выполняется автоматический сценарий: система запускает тестовый контейнер с чистым томом, восстанавливает последний бэкап и проверяет целостность базы данных с помощью встроенных средств SQLite (`PRAGMA integrity_check`). Отчёт о результате восстановления отправляется в тот же канал администраторов.

Дополнительно реализовано точечное резервное копирование перед каждым обновлением планogramмы (через API администратора). При получении команды на изменение планogramмы система автоматически создаёт копию текущей БД с суффиксом `_before_update_YYYYMMDDHHMMSS.db`. Это позволяет откатить ошибочное изменение без обращения к ежедневному бэкапу.

Восстановление из бэкапа выполняется вручную командой администратора. Процесс включает остановку контейнера `bot`, замену файла `planograms.db` на том, перезапуск контейнера и отправку тестового запроса для верификации. Среднее время восстановления (RTO) составляет не более 15 минут, целевая точка восстановления (RPO) – не более 24 часов (при ежедневном бэкапе) или несколько минут (при точечном перед изменением).

Таким образом, система «Retail Vision» готова к производственной эксплуатации с точки зрения сохранности данных и возможности восстановления после сбоев.

В ходе экспериментальной проверки и тестирования разработанного программного обеспечения были зафиксированы количественные показатели точности распознавания товаров и скорости работы нейросетевого модуля. Оптимизированная модель YOLOv8n в формате ONNX продемонстрировала среднюю точность детекции (mAP_{50}) на уровне 88–92%, что полностью удовлетворяет практическим требованиям автоматизированного контроля планogramм для целевых групп товаров «Санта Импэкс». При этом среднее время инференса на один кадр в условиях последовательной обработки одиночных изображений не превысило допустимых лимитов, подтверждая высокую эффективность выбранного runtime-движка.

Интеграция вебхук-эндпоинтов с обратным прокси-сервером и сервером Uvicorn позволила добиться высокой отказоустойчивости при обработке входящего трафика от Telegram API. Применение асинхронного подхода на базе библиотеки `asyncio` для выноса тяжёлых вычислительных задач в фоновый режим исключило риск возникновения сетевых таймаутов. Локальная база данных SQLite успешно справилась с фиксацией промежуточных метрик детекции и логов, подтвердив стабильность разработанной схемы данных при имитации одновременных запросов от нескольких активных мерчендайзеров.

Внедрение разработанного алгоритма сопоставления реальной выкладки с эталонной матрицей расположения продуктов позволило значительно сократить время рутинного аудита торговой точки и снизить влияние человеческого фактора. Накопленная в СУБД ретроспективная статистика закладывает технологический фундамент для дальнейшего масштабирования проекта, включая переход на более мощные архитектуры нейросетей и миграцию на распределённые базы данных для поддержки региональной сети филиалов.

Таким образом, выполненный комплекс работ по развёртыванию системы «Retail Vision» подтвердил её готовность к производственной эксплуатации. Система обеспечивает автоматизированный аудит выкладки продукции Санта Импэкс в торговых точках с использованием обученной модели YOLOv8n, предоставляет мерчендайзеру наглядный отчёт через интерфейс Telegram и демонстрирует устойчивую работу при многопользовательской нагрузке.

6. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ

6.1 Исходные данные для расчета экономического эффекта

Любая разработка программного обеспечения, приложения или их модулей требует финансовых и человеческих ресурсов.

Задачей данного дипломного проекта является разработка нейронной сети типа многослойный персептрон на языке программирования C++.

Разработка программного продукта предусматривает проведение всех стадий проектирования и относится к первой группе сложности.

Последовательность расчетов:

- Расчёт объёма функций программных модулей.
- Расчёт полной себестоимости криптографической защиты.
- Расчёт цены и прибыли по программному продукту.

6.2 Расчет объёма функций программного обеспечения

Общий объём ПО определяется по формуле (6.1), исходя из объёма функций, реализуемых программой.

$$V_0 = \sum_{i=0}^n V_i \quad (6.1)$$

где V_0 – общий объём ПО;

V_i – объём функций ПО;

n – общее число функций.

В том случае, когда на стадии технико-экономического обоснования проекта невозможно рассчитать точный объём функций, то данный объём может быть получен на основании прогнозируемой оценки имеющихся фактических данных по аналогичным проектам, выполненным ранее, или применением нормативов по каталогу функций.

По каталогу функций на основании функций разрабатываемых криптографических модулей определяется общий объём. Также на основе зависимостей от организационных и технологических условий, был скорректирован объём на основе экспертных оценок.

Уточнённый объём ПО определяется по формуле (6.2).

$$V_y = \sum_{i=0}^n V_{yi} \quad (6.2)$$

где V_y – уточнённый объём ПО;

V_{yi} – уточнённый объём отдельной функции в строках исходного кода.

Перечень и объём функций модулей криптографии приведён в таблице 6.1.

Таблица 6.1 – Перечень и объём функций программного обеспечения

Номер функции	Наименование (содержание) функции	Объём функции строк исходного кода	
		По каталогу V_y	Уточнённый V_{yi}
101	Организация ввода информации	150	180
102	Контроль, предварительная обработка и ввод информации	550	450
303	Обработка файлов	1100	570
305	Формирование файла	2460	280
507	Обеспечение интерфейса между компонентами	1820	720
602	Вспомогательные и сервисные программ	580	830
701	Математическая статистика и прогнозирование	4560	2570
703	Расчет показателей	460	830
	ИТОГО	11530	6430

Учитывая информацию, указанную в таблице 6.1, о функциях разрабатываемого программного обеспечения, уточненный объём ПО (V_{yi}) составил 6430 строк исходного кода вместо предполагаемого количества строк 11530.

6.3 Расчет полной себестоимости программного продукта

Стоимостная оценка программного средства у разработчика предполагает составление сметы затрат, которая включает следующие статьи расходов:

- заработную плату исполнителей (основную – ЗПо и дополнительную – ЗПд);
- отчисления на социальные нужды ($P_{соц}$);
- материалы и комплектующие изделия (P_m);
- спецоборудование (P_c);
- машинное время ($P_{мв}$);
- расходы на научные командировки ($P_{нк}$);
- прочие прямые расходы ($P_{пр}$);
- накладные расходы ($P_{нр}$);
- затраты на освоение и сопровождение программного средства (P_o и $P_{со}$).

Полная себестоимость (C_p) разработки ПП рассчитывается как сумма расходов по всем статьям с учетом рыночной стоимости аналогичных продуктов.

Основной статьёй расходов на создание программного продукта является заработная плата проекта (основная и дополнительная) разработчиков (исполнителей) ($ЗП_{осн} + ЗП_{доп}$), в число которых принято включать инженеров-программистов, руководителей проекта, системных архитекторов, дизайнеров, разработчиков баз данных, Web-мастеров и других специалистов, необходимых для решения специальных задач в команде.

Расчёт заработной платы разработчиков ПП начинается с определения:

- Продолжительности времени разработки ($\Phi_{рв}$), которое устанавливается экспертным путем с учётом сложности, новизны ПП и фактически затраченного времени. В данном дипломном проекте $\Phi_{рв} = 70$ дней.

- Количества разработчиков программного обеспечения. В данном дипломном проекте один разработчик категории инженер-программист разряда 8. Заработная плата разработчиков определяется как сумма основной и дополнительной заработной платы всех исполнителей.

Основная заработная плата $ЗП_{осн}$ каждого исполнителя определяется по формуле (6.3):

$$ЗП_{осн} = T_{ст1р} \cdot \frac{T_k}{\Phi_{эфф.р.в.}} \cdot \Phi_{рв} \cdot K_{пр} \quad (6.3)$$

где $T_{ст1р}$ – размер базовой ставки, на текущий момент актуальное значение которой 297 бел.руб.;

T_k – тарифный коэффициент согласно разряду исполнителя;

$\Phi_{эфф.р.в.}$ – среднее количество рабочих дней;

$\Phi_{рв}$ – фонд рабочего времени исполнителя (продолжительность разработки программного модуля), дни;

$K_{пр}$ – коэффициент премии, $K_{пр} = 1,4$.

Тарифный коэффициент согласно 8 разряда инженера-программиста $T_k = 1,57$. Продолжительность разработки программного продукта – 70 дней, количество рабочих дней в месяце на текущий год составляет 22 дня. Рассчитаем основную заработную плату инженера-программиста, согласно формуле (6.3):

$$ЗП_{осн} = 297 \cdot \frac{1,57}{22} \cdot 70 \cdot 1,4 = 2077,11 \text{ (бел.руб.)}$$

Дополнительная заработная плата $ЗП_{доп}$ каждого исполнителя рассчитывается от основной заработной платы по формуле (6.4).

$$ЗП_{доп} = ЗП_{осн} \cdot \frac{Н_{доп.зп}}{100\%} \quad (6.4)$$

где $Н_{доп.зп}$ – надбавка дополнительной заработной платы, равная 15%.

Рассчитаем дополнительную заработную плату инженера-программиста, согласно формуле (4.4):

Рассчитаем заработную плату исполнителей проекта и результаты занесем в таблицу (6.4):

$$ЗП_{доп} = 2077,11 \cdot \frac{15\%}{100\%} = 311,57 \text{ (бел.руб.)}$$

Результаты вычислений внесём в таблицу 6.2.

Таблица 6.2 – Расчет заработной платы

Категория работников	Разряд	T_k	$\Phi_{эфф.р.в.}$	$K_{пр}$	Заработная плата, бел.руб.		
					Осн.	Доп.	Всего
Инженер-программист	8	1,57	70	1,4	2077,11	311,57	2388,68
Итого	—	—	—	—	2077,11	311,57	2388,68

Таким образом, как видно из таблицы, заработная плата инженера-программиста составляет 2388,68 бел. руб.

Отчисления на социальные нужды $P_{\text{соц}}$ определяются по формуле (6.5) в соответствии с действующим законодательством по нормативу (29% – отчисления в ФСЗН + 6% – отчисления по обязательному страхованию).

$$P_{\text{соц}} = (3P_{\text{осн}} + 3P_{\text{доп}}) \cdot \frac{35\%}{100\%} \quad (6.5)$$

Рассчитанное отчисления на социальные нужды:

$$P_{\text{соц}} = (2077,11 + 311,57) \cdot \frac{35\%}{100\%} = 836,04 \text{ (бел.руб.)}.$$

Расходы на материалы и комплектующие $P_{\text{м}}$ отражают расходы на магнитные носители, бумагу, красящие ленты и другие материалы, необходимые для разработки программного продукта. Норма расхода материалов в суммарном выражении определяется в процентах к основной заработной плате (6.6).

$$P_{\text{м}} = 3P_{\text{осн}} \cdot \frac{H_{\text{мз}}}{100\%} \quad (6.6)$$

где $H_{\text{мз}}$ – норма расхода материалов от основной заработной платы, в процентах. Рассчитаем расходы на материалы и комплектующий, приняв $H_{\text{мз}}$ равным 4%:

$$P_{\text{м}} = 2077,11 \cdot \frac{4\%}{100\%} = 83,08 \text{ (бел.руб.)}$$

Расходы на спецоборудование $P_{\text{с}}$ включают затраты на приобретение технических и программных средств специального назначения, необходимых для разработки методического пособия, включая расходы на проектирование, изготовление, отладку и другое.

В данном дипломном проекте для разработки нейронной сети приобретение какого-либо спецоборудования не предусматривалось. Так как спецоборудование не было приобретено, расходы равны нулю.

Расходы на машинное время $P_{\text{мв}}$ включают оплату машинного времени, необходимого для разработки и отладки программного продукта. Они определяются в машино-часах по нормативам на 100 строк исходного кода машинного времени. $P_{\text{мв}}$ определяется по формуле (6.7).

$$P_{\text{мв}} = C_{\text{ми}} \cdot \frac{V_0}{100} \cdot H_{\text{мв}} \quad (6.7)$$

где $C_{\text{ми}}$ – цена одного машинного часа (7 бел. руб.);

V_0 – уточнённый общий объём машинного кода;

$H_{\text{мв}}$ – норматив расхода машинного времени на отладку 100 строк кода в машино-часах. Принимается в размере 0,6.

Рассчитаем расходы на машинное время:

$$P_{\text{мв}} = 7 \cdot \frac{6430}{100} \cdot 0,6 = 270,06 \text{ (бел. руб.)}$$

Расходы по статье «Научные командировки» $P_{\text{нк}}$ берутся либо по смете научных командировок, разрабатываемой на предприятии, либо в процентах от основной заработной платы исполнителей (10-15%). Так как в данном проекте научные командировки не предусмотрены, данная статья не рассчитывается.

Прочие затраты $P_{\text{пр}}$ включают затраты на приобретение специальной технической

информации и специальной литературы. Определяются по нормативу в процентах к основной заработной плате исполнителей. Так как специальная научно-техническая информация и специальная литература не приобреталась, то данная статья не рассчитывается.

Затраты на накладные расходы $P_{\text{нр}}$ связаны с содержанием вспомогательных хозяйств, опытных производств, а также с расходами на общехозяйственные нужды. Определяется по нормативу в процентах к основной заработной плате по формуле (6.8).

$$P_{\text{нр}} = \frac{N_{\text{нр}}}{100\%} \cdot 3П_{\text{осн}} \quad (6.8)$$

где $N_{\text{нр}}$ – норматив накладных расходов.

В данном дипломном проекте норматив накладных расходов равен 75%, поэтому затраты на накладные расходы согласно формуле (6.8) равны:

$$P_{\text{нр}} = \frac{75\%}{100\%} \cdot 2077,11 = 1557,83 \text{ (бел.руб.)}.$$

Сумма вышеперечисленных расходов по статьям на программный продукт служит исходной базой для расчёта затрат на освоение и сопровождение программного продукта. Они рассчитываются по формуле (6.9):

$$C3 = 3П_{\text{осн}} + 3П_{\text{доп}} + P_{\text{соц}} + P_{\text{м}} + P_{\text{с}} + P_{\text{мв}} + P_{\text{нк}} + P_{\text{пр}} + P_{\text{нр}} \quad (6.9)$$

тогда

$$C3 = 2077,11 + 311,57 + 836,04 + 83,08 + 270,06 + 1557,83 = 5135,69 \text{ (бел.руб.)}.$$

Организация-разработчик участвует в освоении программного продукта и несёт соответствующие затраты, на которые составляется смета, оплачиваемая заказчиком по договору. Затраты на освоение P_o определяются по установленному нормативу от суммы затрат по формуле (6.10).

$$P_o = C3 \cdot \frac{N_o}{100\%} \quad (6.10)$$

где $C3$ – сумма вышеперечисленных расходов по статьям на разработку программного продукта;

N_o – установленный норматив затрат на освоение. Для данного дипломного проекта принимается равным 8%.

Рассчитаем затраты на освоение продукта:

$$P_o = 5135,69 \cdot \frac{8\%}{100\%} = 410,86 \text{ (бел.руб.)}$$

Организация-разработчик осуществляет сопровождение программного продукта и несёт расходы, которые оплачиваются заказчиком в соответствии с договором и сметой на сопровождение. Расходы на сопровождение $P_{\text{со}}$ рассчитываются по формуле (6.11).

$$P_{\text{со}} = C3 \cdot \frac{N_{\text{со}}}{100\%} \quad (6.11)$$

где $N_{\text{со}}$ – установленный норматив затрат на сопровождение программного продукта. Для данного дипломного проекта принимается равным 7%.

Рассчитаем расходы на сопровождение:

$$P_{\text{со}} = 5135,69 \cdot \frac{7\%}{100\%} = 359,50 \text{ (бел.руб.)}$$

Полная себестоимость (СП) разработки программного продукта рассчитывается как сумма расходов по всем статьям. Она определяется по формуле (6.12).

$$СП = СЗ + Р_o + Р_{co} \quad (6.12)$$

Рассчитанная полная себестоимость продукта:

$$СП = 5135,69 + 410,86 + 359,50 = 5906,05 \text{ (бел.руб.)}$$

Результаты вычислений занесём в таблицу 6.3.

Таблица 6.3 – Себестоимость программного продукта

Наименование статей затрат	Норматив, %	Сумма затрат, бел.руб.
Заработная плата, всего	–	2388,68
Основная заработная плата	–	2077,11
Дополнительная заработная плата	–	311,57
Отчисления на социальные нужды	35	836,04
Спецоборудование	Не применялось	–
Материалы и комплектующие изделия	4	83,08
Машинное время	–	270,06
Научные командировки	Не планировались	–
Прочие затраты	Не применялись	–
Накладные расходы	75	1557,83
Сумма затрат	–	5135,69
Затраты на освоение	8	410,86
Затраты на сопровождение	7	359,50
Полная себестоимость	–	5906,05

В результате всех расчётов полная себестоимость программного продукта составила 5906,05 бел.руб.

6.4 Расчет цены и прибыли по программному продукту

Для определения цены программного продукта необходимо рассчитать плановую прибыль П, которая рассчитывается по формуле (6.13).

$$П = СП \cdot \frac{R}{100\%} \quad (6.13)$$

где СП – полная себестоимость программного модуля, бел. руб;

R – уровень рентабельности программного модуля.

Рассчитаем прибыль от реализации:

$$П = 5906,05 \cdot \frac{23\%}{100\%} = 1358,39 \text{ (бел.руб)}$$

После расчета прибыли от реализации по формуле (6.14) определяется прогнози-

руемая цена программного продукта без налогов Π_{Π} .

$$\Pi_{\Pi} = \text{СП} + \Pi \quad (6.14)$$

Рассчитаем цену программного продукта без налогов:

$$\Pi_{\Pi} = 5906,05 + 1358,39 = 7264,44 \text{ (бел.руб.)}$$

Отпускная цена Π_0 (цена реализации) программного продукта включает налог на добавленную стоимость и рассчитывается по формуле (6.15).

$$\Pi_0 = \text{СП} + \Pi + \text{НДС}_{\text{пп}} \quad (6.15)$$

где $\text{НДС}_{\text{пп}}$ – налог на добавленную стоимость для программного продукта. Для данного программного продукта $\text{НДС}_{\text{пп}}$ рассчитывается по формуле (6.16).

$$\text{НДС}_{\text{пп}} = \Pi_{\Pi} \cdot \frac{\text{НДС}}{100\%} \quad (6.16)$$

где НДС – налог на добавленную стоимость. В настоящее время он составляет 20%.

Рассчитаем $\text{НДС}_{\text{пп}}$:

$$\text{НДС}_{\text{пп}} = 7264,44 \cdot \frac{20\%}{100\%} = 1452,89 \text{ (бел.руб.)}$$

Рассчитаем отпускную цену:

$$\Pi_0 = 5906,05 + 1358,39 + 1452,89 = 8717,33 \text{ (бел.руб.)}$$

Прибыль от реализации программного продукта за вычетом налога на прибыль является чистой прибылью. Чистая прибыль остаётся организации-разработчику и представляет собой экономический эффект от создания нового программного продукта. Она рассчитывается по формуле (6.17).

$$\text{ПЧ} = \Pi \cdot \left(1 - \frac{H_{\Pi}}{100\%}\right) \quad (6.17)$$

где H_{Π} – ставка налога на прибыль. В настоящее время он равен 20%.

Рассчитаем чистую прибыль:

$$\text{ПЧ} = 1358,39 \cdot \left(1 - \frac{20\%}{100\%}\right) = 1086,71 \text{ (бел.руб.)}$$

Результаты расчётов цены и прибыли по программному продукту сведены в таблицу 6.4.

Таблица 6.4 – Расчёт отпускной цены и чистой прибыли программного продукта

Наименование статей затрат	Норматив, %	Сумма затрат, бел.руб.
Полная себестоимость	–	5906,05
Прибыль	23	1358,39
Цена без НДС	–	7264,44
НДС	20	1452,89
Отпускная цена	–	8717,33
Налог на прибыль	20	271,68
Чистая прибыль	–	1086,71

В ходе произведенных расчетов определены основные экономические показатели: полная себестоимость – 5906,05 бел.руб.; прогнозируемая цена – 8717,33 бел. руб.; чистая прибыль – 1086,71 бел.руб.

Разработанный программный продукт не имеет конкурентов, предлагающих свои решения по более низким ценам. Несмотря на сравнительно низкую стоимость разрабатываемой нейронной сети, её функциональность и качество остаются на высоком уровне. Стоимость системы, разработанной в ходе дипломного проектирования, является приемлемой для основных потребителей, в лице промышленных предприятий. Основным плюсом для приобретателя нейронной сети является возможность интеграции нейронной сети в иные системы, а также невысокие системные требования, предъявляемые к машине, на которой запускается система. Таким образом, рассчитанная отпускная цена на веб-приложение, разрабатываемое в рамках данного дипломного проекта, составляет 8717,33 белорусских рубля и является конкурентоспособной. При расчете цены учтены отчисления в фонд социальной защиты, а также налоги, необходимые к уплате.

ЗАКЛЮЧЕНИЕ

В рамках данного дипломного проекта было разработано программное обеспечение для автоматизированной проверки выкладки товаров на торговых полках, включающее нейросетевой модуль на основе архитектуры YOLOv8 и Telegram-бот для полевых сотрудников компании «Санта Импэкс».

В процессе выполнения дипломного проекта были решены следующие задачи:

1. Проведён анализ предметной области мерчендайзинга и требований компании «Санта Импэкс» к контролю соответствия выкладки товаров планограммам.
2. Выполнен обзор существующих аналогов систем компьютерного зрения для розничной торговли и обоснован выбор архитектуры YOLOv8 в качестве базовой модели обнаружения объектов.
3. Сформирован и размечен набор данных для обучения модели с использованием платформы Roboflow; применены методы аугментации для повышения обобщающей способности нейронной сети.
4. Выполнено обучение и дообучение модели YOLOv8 на собственном датасете; проведена оценка качества по метрикам mAP, Precision и Recall.
5. Разработан Telegram-бот для полевых сотрудников, обеспечивающий приём фотографий торговых полок, их передачу на сервер анализа и возврат результатов проверки без установки дополнительного программного обеспечения.
6. Спроектирована и реализована серверная часть системы, включающая модуль инференса нейронной сети на основе ONNX Runtime и базу данных для хранения результатов проверок.
7. Реализован алгоритм сопоставления обнаруженных товарных позиций с эталонной планограммой и формирования отчёта об отклонениях.
8. Проведено тестирование всех компонентов системы: нейросетевого модуля, Telegram-бота и серверной логики.

Разработанное программное обеспечение удовлетворяет всем поставленным требованиям, а именно:

1. Автоматическое обнаружение товарных позиций на фотографиях торговых полок в режиме реального времени.
2. Сравнение фактической выкладки с эталонной планограммой и выявление несоответствий.
3. Доступность системы через мессенджер Telegram без необходимости установки дополнительного ПО на устройство мерчендайзера.
4. Хранение истории проверок и формирование отчётности по торговым точкам.
5. Устойчивость системы к сбоям и корректная обработка ошибочных входных данных.

Проведённое тестирование подтвердило корректность работы всех функциональных модулей и соответствие результатов распознавания заданным показателям качества.

Для разработанного программного обеспечения был осуществлён расчёт экономической эффективности, определена полная себестоимость проекта, отпускная цена и чистая прибыль разработчика от реализации программного продукта.

СПИСОК ЛИТЕРАТУРЫ

1. Редмон, Дж. Вы смотрите только один раз: единая, реалистичная детекция объектов в реальном времени / Дж. Редмон, С. Дивала, Р. Гиршик, А. Фархади // Компьютерное зрение и распознавание образов. – 2016. – С. 779–788.
2. Ultralytics YOLOv8 – документация и руководство пользователя [Электронный ресурс] / Ultralytics Inc. – 2023. – Режим доступа: <https://docs.ultralytics.com/>. – Дата доступа: 10.02.2025.
3. Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль ; пер. с англ. А. А. Слинкина. – 2-е изд., испр. – М. : ДМК Пресс, 2018. – 652 с. : цв. ил.
4. Головкин, В. А. Нейросетевые технологии обработки данных : учеб. пособие / В. А. Головкин, В. В. Краснопрошин. – Минск : БГУ, 2017. – 263 с. – (Классическое университетское издание).
5. Шолле, Ф. Глубокое обучение на Python / Ф. Шолле ; пер. с англ. А. А. Слинкина. – 2-е изд. – СПб. : Питер, 2023. – 400 с. : ил.
6. Николенко, С. И. Глубокое обучение / С. И. Николенко, А. А. Кадурын, Е. О. Архангельская. – СПб. : Питер, 2020. – 480 с. : ил. – (Серия «Библиотека программиста»).
7. Roboflow – платформа для разметки и аугментации датасетов компьютерного зрения [Электронный ресурс] / Roboflow Inc. – 2024. – Режим доступа: <https://docs.roboflow.com/>. – Дата доступа: 18.02.2025.
8. ONNX Runtime – кроссплатформенный инференс нейронных сетей [Электронный ресурс] / Microsoft Corporation. – 2024. – Режим доступа: <https://onnxruntime.ai/docs/>. – Дата доступа: 25.02.2025.
9. Telegram Bot API – официальная документация [Электронный ресурс] / Telegram Messenger Inc. – 2024. – Режим доступа: <https://core.telegram.org/bots/api>. – Дата доступа: 01.03.2025.
10. Trax Retail – платформа для аналитики розничных полок [Электронный ресурс] / Trax Retail Ltd. – 2024. – Режим доступа: <https://traxretail.com/>. – Дата доступа: 05.03.2025.
11. Vispera – решения для распознавания товаров на полках [Электронный ресурс] / Vispera Information Technologies. – 2024. – Режим доступа: <https://vispera.co/>. – Дата доступа: 05.03.2025.
12. Бурков, А. Инженерия машинного обучения / А. Бурков ; пер. с англ. А. А. Слинкина. – М. : ДМК Пресс, 2022. – 306 с. : цв. ил.
13. Эверингем, М. Задача визуального распознавания объектов в рамках вызова PASCAL / М. Эверингем [и др.] // Международный журнал компьютерного зрения. – 2010. – Т. 88, № 2. – С. 303–338.
14. Шалобыта, Н. Н. Методы и средства автоматизации в торговле : пособие / Н. Н. Шалобыта. – Брест : БрГТУ, 2021. – 158 с.
15. Колесникович, А. В. Применение свёрточных нейронных сетей для анализа изображений в задачах ритейла / А. В. Колесникович, Д. С. Лукашевич // Вестник БрГТУ. Физика, математика, информатика. – 2022. – № 3. – С. 64–69.